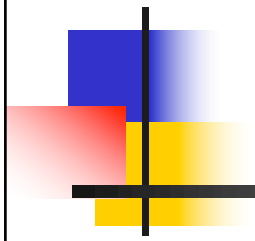
Réseaux Informatiques



CM 6 : Couche Transport

Responsable : Mme Gladys DIAZ

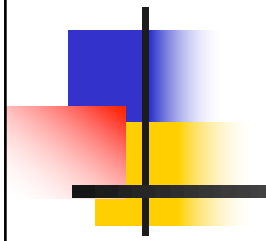
L2TI – E218

E-mail : gladys.diaz@univ-paris13.fr

Formation : Master Informatique

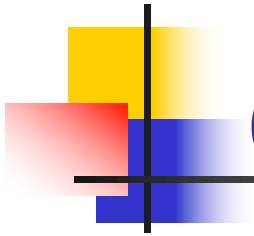
UNIVERSITÉ SORBONNE PARIS NORD

Institut GALILÉE
Université Sorbonne Paris Nord



☞ Séance 6

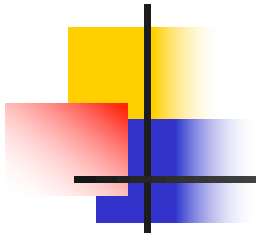
Couche Transport (1/2)



Couche Transport -Plan

Introduction à la couche Transport

1. Présentation de la couche transport
2. Entité transport, Services, Primitives de transport
3. Protocoles de transport
4. Adressage, multiplexage
5. Contrôle de flux et mémorisation
6. Rappels pile de protocoles Internet



1. Couche Transport

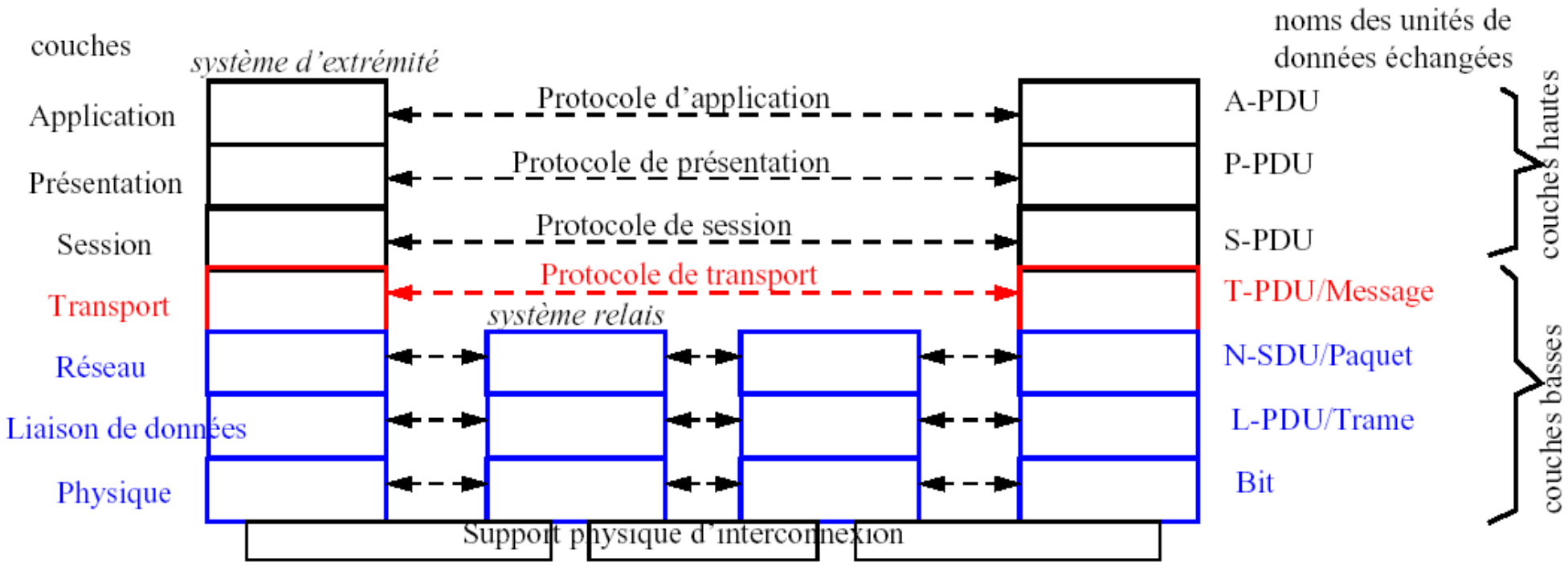
Introduction

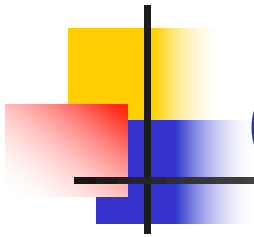
- La couche Transport du modèle OSI :
  - quatrième couche du modèle de référence
  - dernière couche des services de bas niveau

- La couche Transport assure la charnière entre les aspects applicatifs et les aspects liés à la transmission de données sur un réseau informatique.

Couche Transport

Introduction





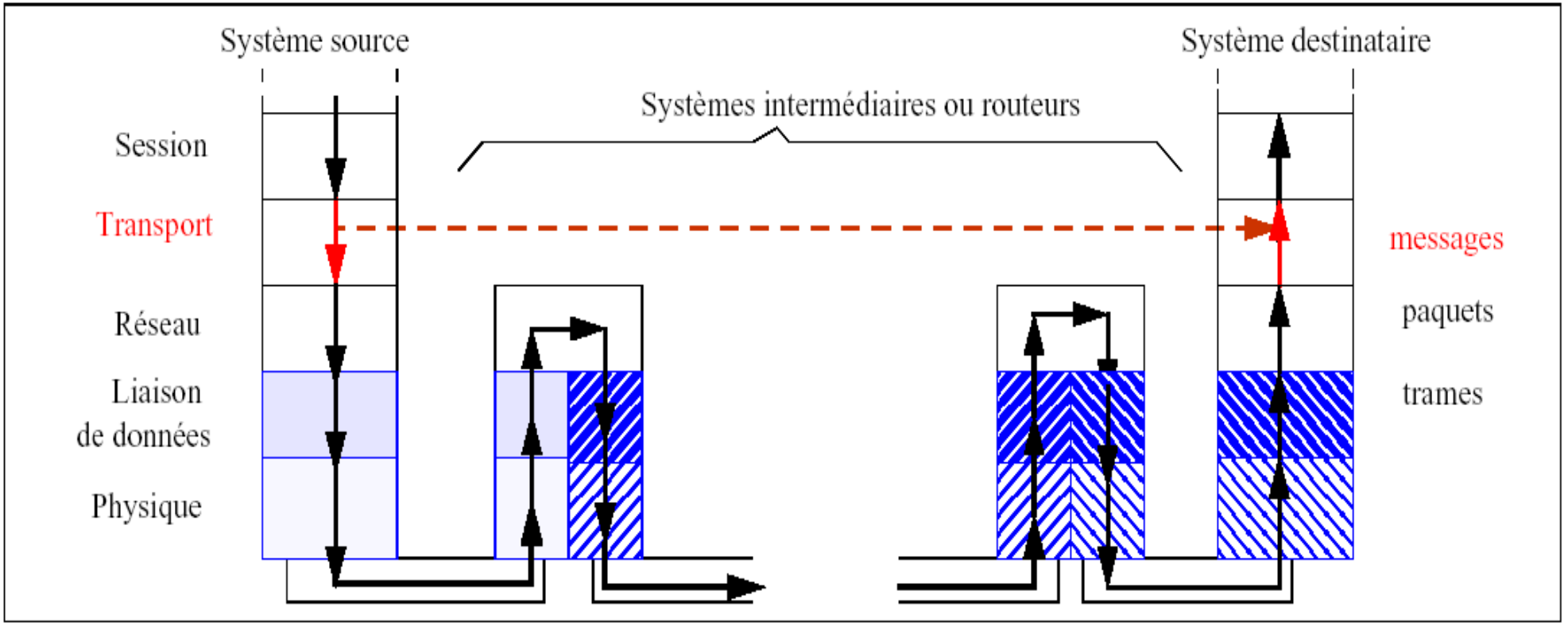
Couche Transport

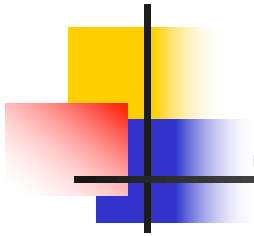
Introduction

- Le rôle principal de la couche transport
- La couche Transport assure le transport de bout en bout des données d'une façon sûre et efficace.
  - transfert d'informations entre systèmes d'extrémité
  - de manière efficace, fiable et économique,
  - indépendamment de la nature des réseaux sous-jacents

Couche Transport

- Le logiciel et/ou le matériel qui assure cette fonction est appelé l'entité de transport





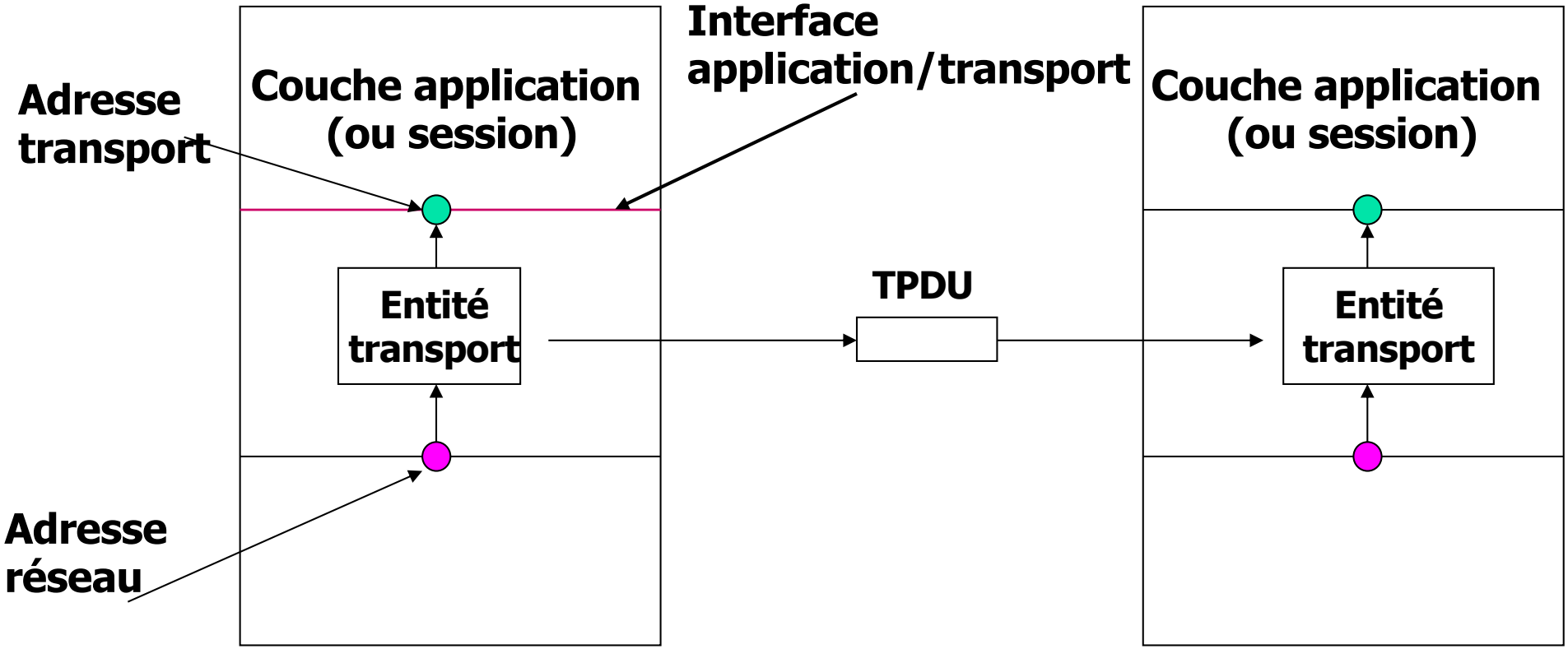
2. Entité Transport

Introduction

- Une entité de transport peut être dans le noyau du système d'exploitation, dans un processus utilisateur, dans un paquetage de bibliothèque ou encore dans la carte d'accès au réseau

- Dans certaines cas, l'opérateur de réseau peut même offrir un service de transport fiable, auquel cas l'entité de transport se situe sur une machine interface spéciale, à la frontière du sous-réseau auquel les hôtes sont connectés

Entité Transport

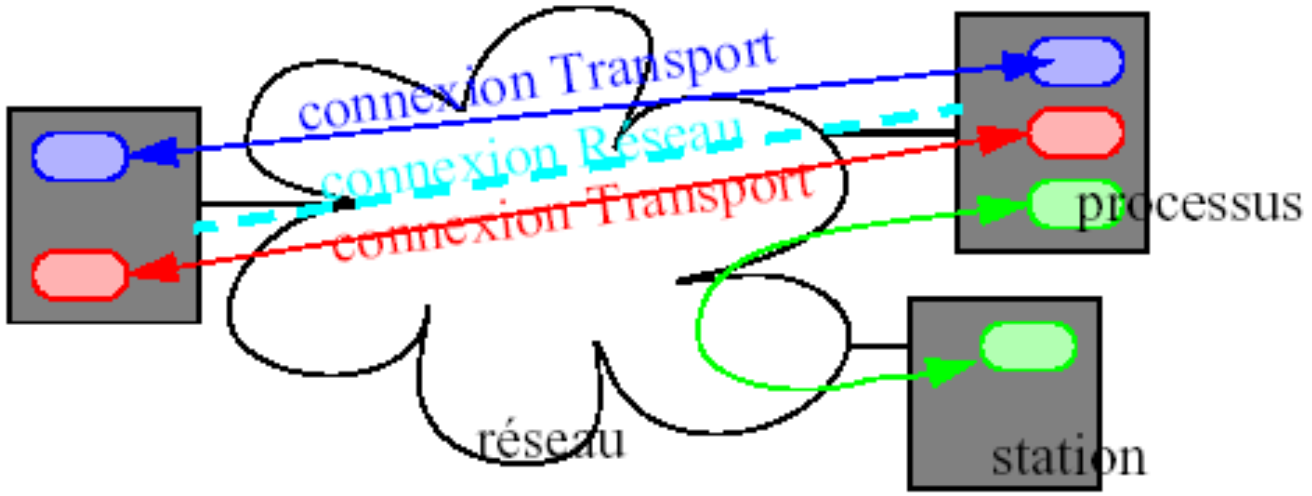


Les couches réseau, transport et application

Entité Transport

Fonctions

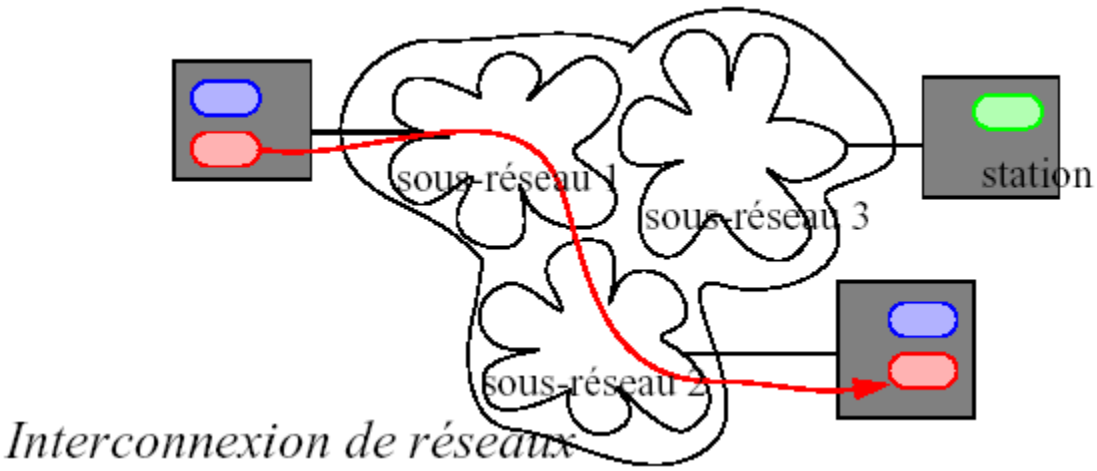
- Les entités communicantes ne sont plus des stations mais des processus :
  - adressage spécifique
  - multiplexage des connexions Transport sur une connexion Réseau reliant les stations supportant les processus.



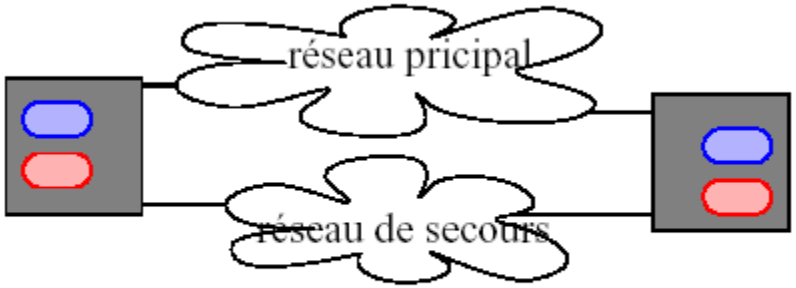
Couche Transport

Fonctions

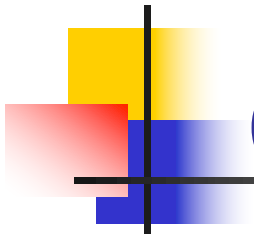
- La connectivité doit être assurée et maintenue en dépit de la multiplicité des réseaux utilisés pour l'acheminement.



Interconnexion de réseaux



Redondance



Couche Transport

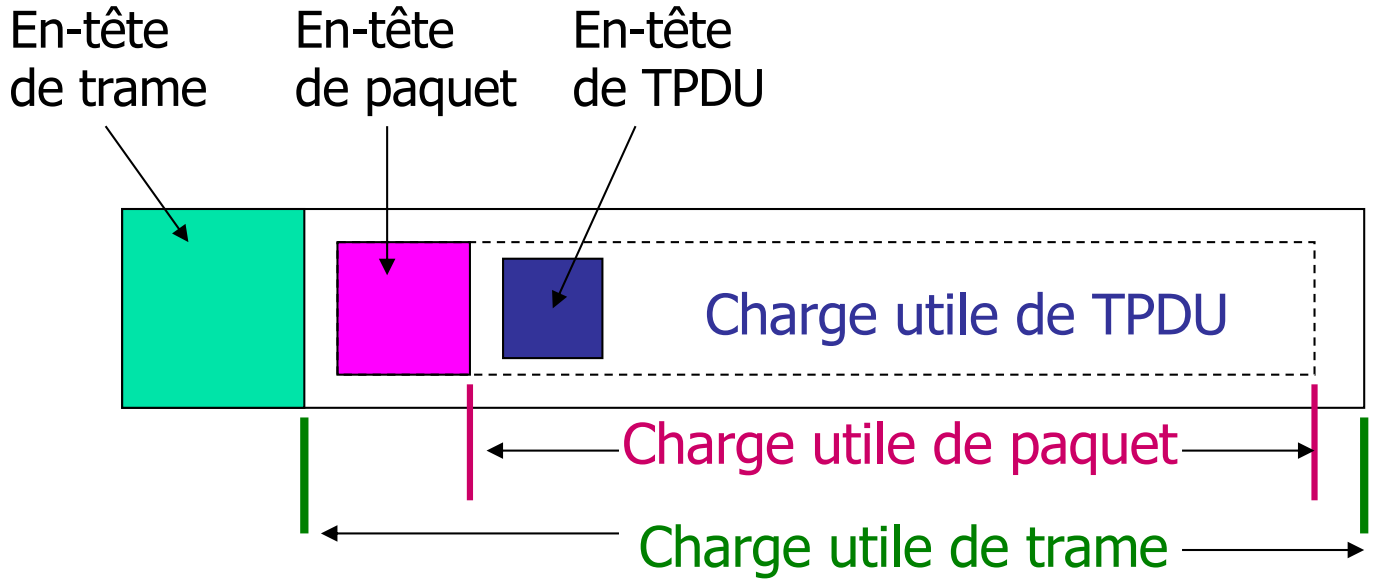
Fonctions

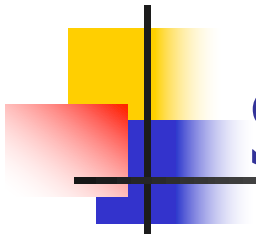
- La couche Transport peut offrir les deux modes classiques de service :
  - le mode connecté et le mode non connecté
- Deux types de services de transmission :
  - flux continu de données (“stream oriented service”) : un seul T_SDU !
  - séquence d’unités de données (“record oriented service”) : une suite de T_SDU
- Le message : c’est l’unité de données de transfert de niveau Transport (T-PDU).
  - Il est généralement de taille variable et composée d’un champ d’entête et d’un champ d’information.

Couche Transport

TPDU

- TPDU : Transport Protocol Data Unit
  - messages échangés entre deux entités de transport

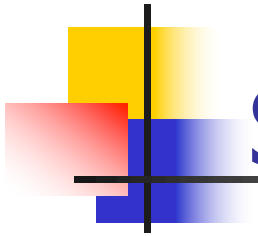




Service Transport

Pourquoi une autre couche de bout-en-bout ?

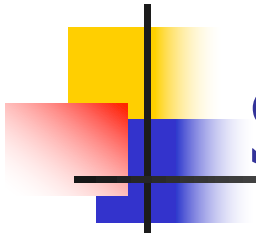
- construction du service attendu par les applications distinct de celui fournis par la couche réseau
  - mode connecté fiable sur un service réseau datagramme non-fiable (exemple : TCP/IP)
- séparation des entités réseaux et applicatives
  - indépendance des primitives de transports
  - mécanismes seulement dans les hôtes d'extrémités
- séparation du service fournis par l'opérateur du réseau et l'utilisateur



Service Transport

Qualité de Service

- La couche Transport optimise l'utilisation des ressources disponibles (du Réseau) pour atteindre (et maintenir) la qualité de service (de transmission de données) souhaitée par les entités supérieures (applicatives).
  - Couche transport complémente ou améliore de la Qualité de Service (QoS) fournis par la couche réseau

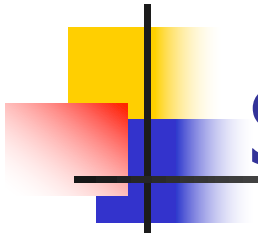


Service Transport

Qualité de Service

- La notion de QoS peut être définie par l'ensemble des paramètres (délai de transit, débit, etc.) caractérisant les besoins des entités applicatives.

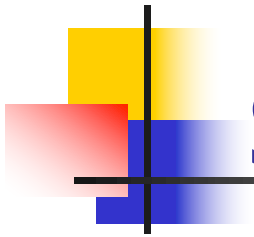
- La valeur des paramètres peut être négociée :
  - Lors de la phase d'établissement de la connexion, en fonction :
    - des caractéristiques des réseaux sous-jacents ($\Downarrow$)
    - des demandes des entités supérieures locales et distantes ($\Uparrow$)
    - des mécanismes protocolaires disponibles ($\Leftrightarrow$)



Service Transport

Qualité de Service

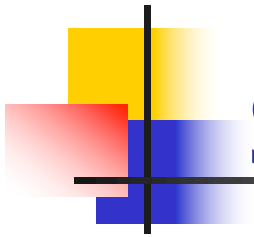
- Paramètres potentiellement associés à la couche transport (généralement négociés) :
  - délais d'établissement d'une connexion
    - durée qui s'écoule entre l'émission d'une demande de connexion par l'utilisateur et la réception de sa confirmation
  - taux d'échec d'établissement d'une connexion
    - Mesure le risque qu'une connexion ne puisse s'établir dans un délai maximum défini
  - débit de la liaison (dans chaque direction)
    - Nombre d'octets utiles qui peuvent être transférés en une seconde



Service Transport

Qualité de Service

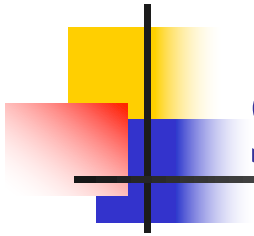
- temps de transit (estimé dans chaque direction)
  - temps écoulé entre le moment où l'utilisateur du service de transport envoi un message et celui où l'entité de transport réceptrice le reçoit effectivement
- taux d'erreur résiduel (en théorie il devrait être 0)
  - Rapport du nombre de messages perdus ou mal transmis ou nombre total de messages transmis pendant une période donnée



Service Transport

Qualité de Service

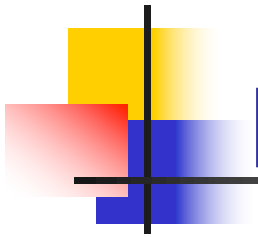
- sécurité (écoutes; modification...)
  - Possibilité laissé à l'utilisateur d'interdire à un terminal tiers l'intrusion ou l'interférence sur sa ligne pour lire ou modifier les données transmises
- politique de service (priorité...)
  - Permet à l'utilisateur de privilégier certaines connexions par rapport à des autres



Service Transport

Qualité de Service

- Le service transport permet de distinguer pour ces paramètres 3 types de valeurs (dans le modèle OSI) : préféré, acceptable et inacceptable, qui sont choisis lors de l'établissement de la connexion
  - Négociation d'options, inchangées pendant toute la durée de la connexion
  - Les opérateurs de réseaux fixent leur prix en fonction de la qualité de service
- C'est le rôle de la couche transport de surveiller ces paramètres, et en fonction du type de service produit par la couche réseau, de déterminer si elle est en mesure de respecter ses engagements



Primitives de Transport

- Propres à chaque service de transport
- Similaires à celle de la couche réseau mais détachées des technologies réseaux
  - Le service réseau n'est utilisé que par des entités de transport
  - En revanche nombreux sont les programmes (programmeurs) qui se servent de primitives de transport

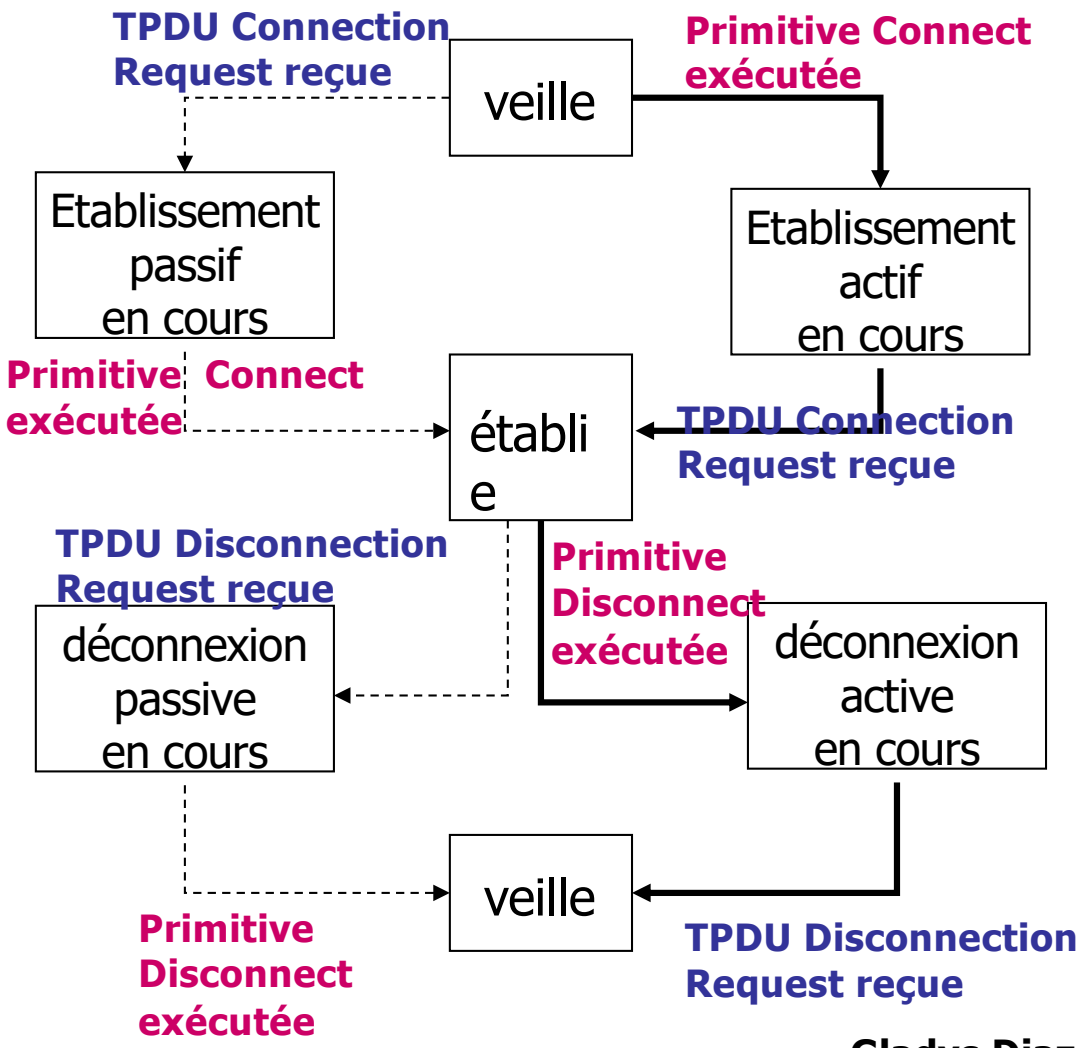
Primitives de Transport

Exemple de primitives pour un service simple en mode connecté fiable

- permet aux programme d'application d'établir, d'utiliser et de libérer des connexions

Primitive	TPDU envoyée	Description
LISTEN	Rien	Se bloque jusqu'à ce qu'un processus se connecte
CONNECT	CONNECTION REQUEST	Tentative d'établissement de connexion
SEND	DATA	Envoi d'information
RECEIVE	Rien	Se bloque jusqu'à une TPDU DATA
DISCONNECT	DISCONNECTION REQUEST	Ce côté veut libérer la connexion

Primitives de Transport



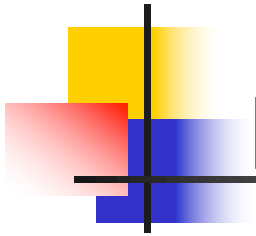
Mode Client-Serveur

Diagramme d'état pour la gestion d'une connexion

- **Transitions déclenchées par des primitives locales ou des arrivées de TPDU**
- **TPDU acquittée séparément**
- **Déconnexion symétrique**

- **états du client**

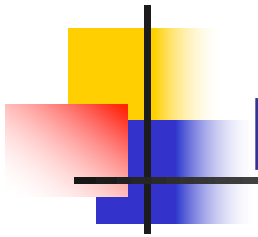
- **états du serveur**



Primitives Transport : exemple TCP

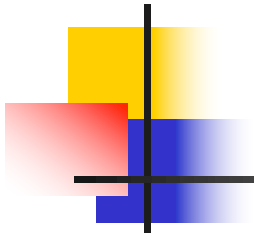
- Primitives pour TCP utilisées par les UNIX BSD*
  - SOCKET (création d'un nouveau point terminal)
    - Pas d'adresse initial, jusqu'à l'exécution d'un BIND
    - Allocation d'un espace de tables dans l'entité transport
    - Spécification, à travers des paramètres, de : format d'adressage utilisé, type de service désiré (ex flots d'octets fiable) et protocole
  - BIND (attache une adresse au socket)
  - LISTEN (acceptation des connexions avec file d'attente)
    - allocation de l'espace pour mettre en file d'attente les appels entrants (dans le cas où plusieurs clients veulent se connecter au même temps)

* Berkeley Software Distribution



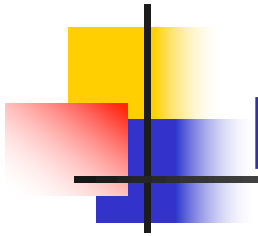
Primitives Transport : exemple TCP

- ACCEPT (acceptation bloquante)
  - Le serveur se bloque dans l'attente d'une connexion entrant
- CONNECT (établissement actif de connexion)
  - Bloque l'appelant et démarre le processus de connexion
  - Lorsque le client reçoit le TPDU approprié du serveur on débloque le processus client et on établit la connexion
- SEND (envoi de données sur une connexion)
- RECEIVE (réception de donnée d'une connexion)
- CLOSE (terminaison d'une connexion)



Primitives Transport : exemple TCP

- **Socket**
  - Un "socket" est défini par l'association des adresses Internet source, destinataire, ainsi que les deux adresses de port à chaque bout.
  - Une connexion nécessite la mise en place de deux sockets
  - Exemple type de socket :
    - (10.0.0.2, TCP, 1055) <--> (10.0.0.1, TCP, 80)

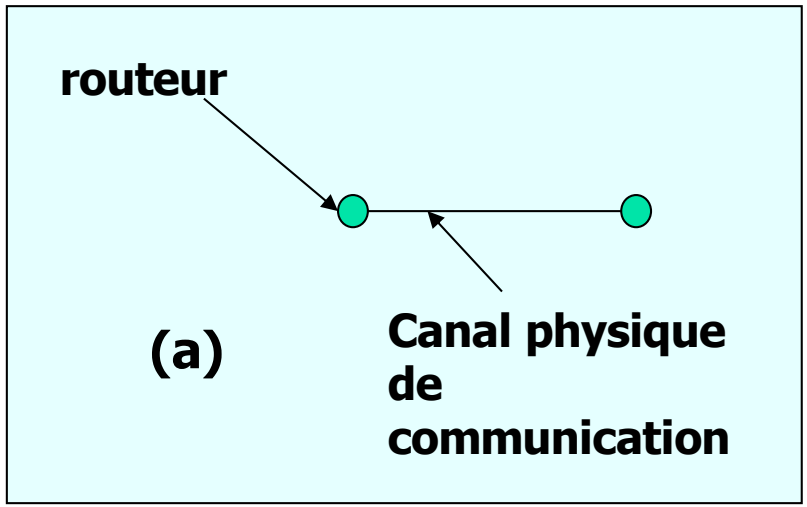


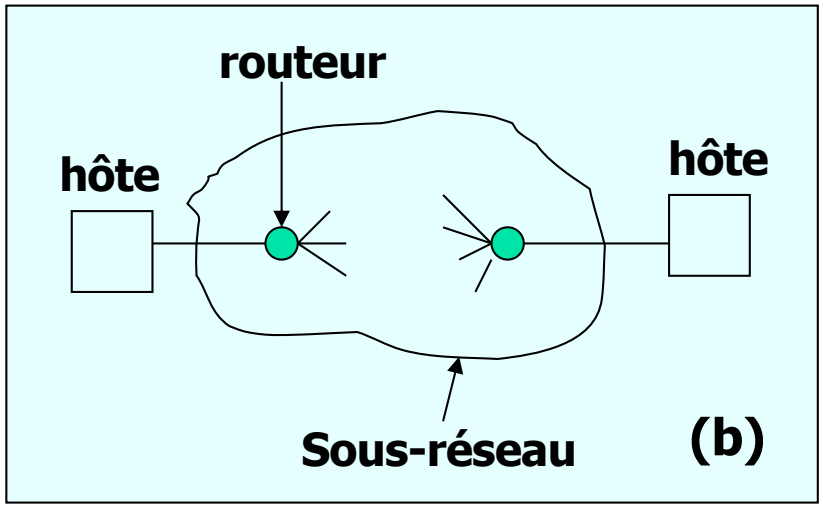
Primitives Transport : Exemple TCP

- Serveur :
  - SOCKET –> BIND –> (LISTEN) –> ACCEPT …[SEND,RECEIVE]*… –> CLOSE
- Client :
  - SOCKET –> (BIND) –> CONNECT …[SEND,RECEIVE]*… –> CLOSE

3. Protocoles de Transport

- Le service de transport est fournis grâce à un protocole de transport utilisé entre les deux entités en communication
- similitude à la couche liaison (a) mais couche transport (b) avec :
  - adressage explicite
  - gestion de connexion complexe
  - mémoire, déséquencement, variabilité...

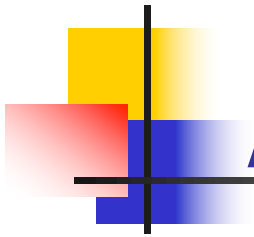




4. Adressage : TSAP

Adressage

- Double besoin pour les adresses :
  - identifier les processus dans les stations
    - durée de vie égale à celle du processus : réutilisation des mêmes adresses
    - connaissance de ces adresses limitée aux intervenants
  - identifier des services généraux (“Well known generic service address”)
    - durée de vie permanente
    - connaissance totale étendue à tous les acteurs potentiels
- L’adresse Transport complète l’adressage Réseau.
- L’adressage s’effectue via les points d’accès au service Transport :
  - TSAP (“Transport Service Access Point”) auxquels chaque processus vient se connecter et où il attend une demande de connexion.



Adressage : TSAP

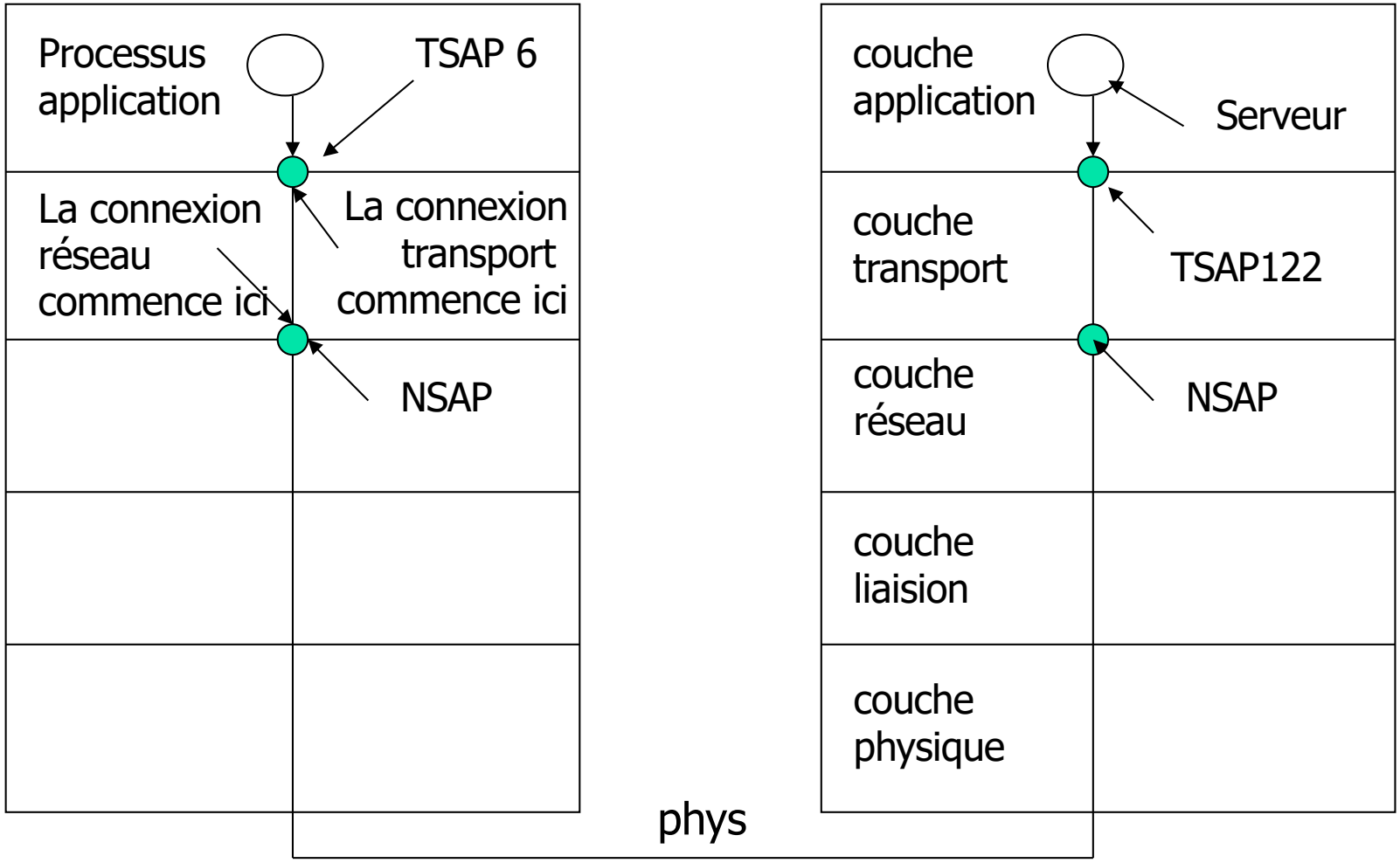
- **TSAP : Transport Service Access Point**
  - Identification des applications en communication
  - Adresses de transport auxquels les processus peuvent être à l'écoute d'éventuelles demandes de connexion
    - Adresse IP + Port (Internet)
    - SAP-AAL (ATM)
    - …
  - Les points terminaux analogues dans la couche réseau (adresses couche réseau) :
    - NSAP (Network Service Access Point)
    - Adresses IP
    - Il peut y avoir plusieurs ports attachés à un NSAP

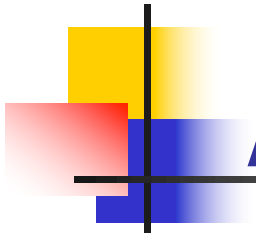
Adressage : TSAP

- **Scénario de connexion :**
  1) Serveur de date sur host2 associé au TSAP 122
  2) Client sur host1 s'associe localement au TSAP 6 (source) et demande le TSAP 122 sur host2 (destination)
  3) L'entité de transport de 1 sélectionne une adresse réseau sur sa machine (s'il y a plus d'une) et sur la machine destination, et établit une connexion réseau entre elles (source = host1, destination = host2)
  4) Le client demande d'établissement d'une connexion au niveau transport entre TSAP 6 et TSAP 122
  5) L'entité transport sur 2 demande au Serveur de date s'il veut la validation de cette connexion ...

Adressage : TSAP

TSAP / NSAP

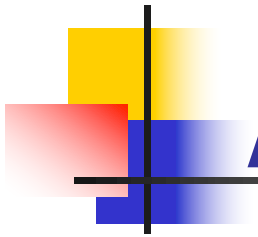




Adressage : TSAP

Exemple de connexion

- Problèmes
  - Comment savoir le TSAP d'un service ?
    - Well known (services adresse TSAP fixe)
    - Annuaire (serveur de noms)
      - Envoyer le nom du service demandé vers un « serveur de noms » qui répond avec le TSAP du service
  - Comment gérer de nombreux serveurs en attente :
    - Utilisation d'un **protocole de connexion initial (préconnexion)**
      - Pas d'attribution d'adresse définie à chaque serveur potentiel, mais chacun a un serveur de processus spécial (**process server :** inetd UNIX) qui agit comme délègue (proxy) de serveurs peu utilisés



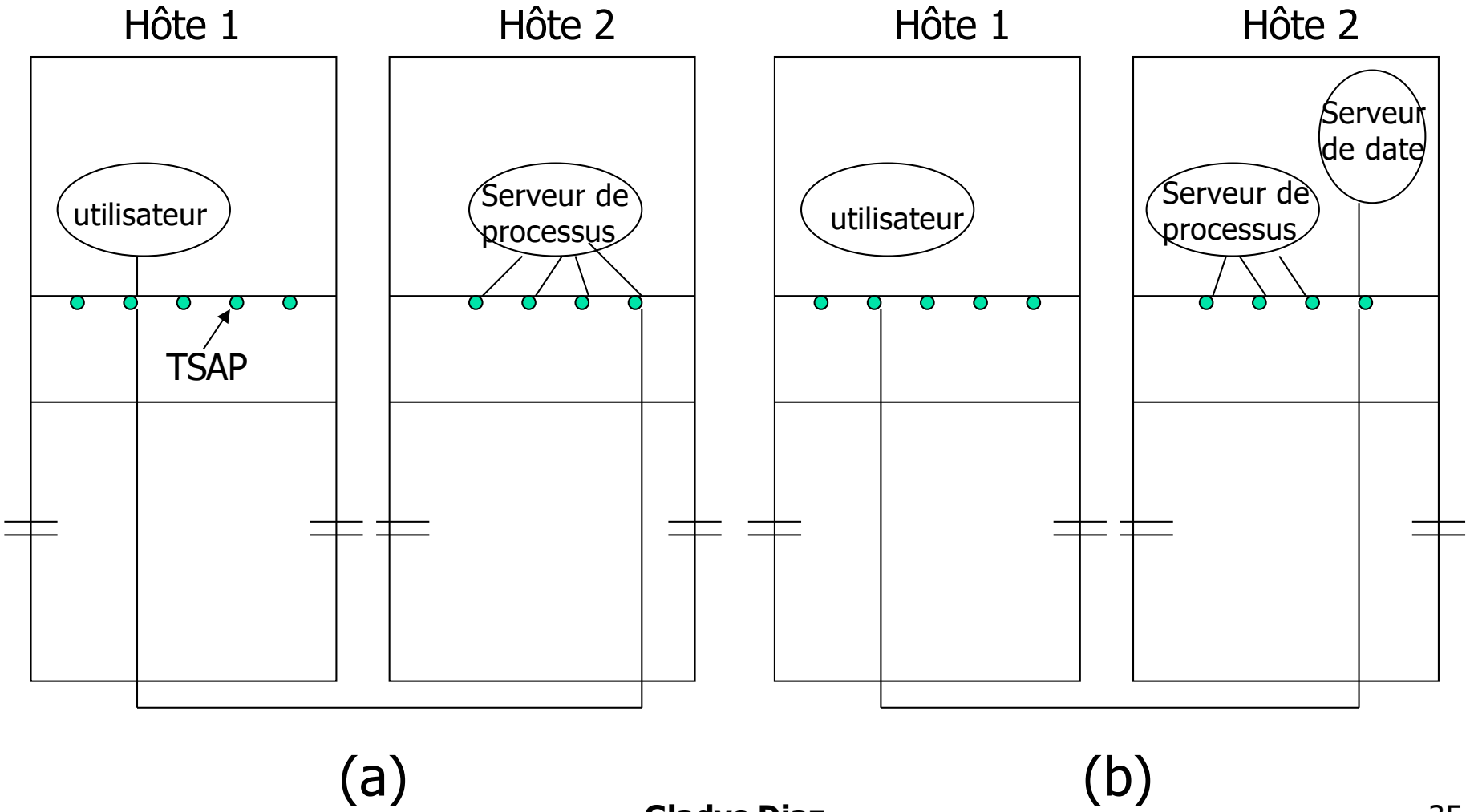
Adressage : TSAP

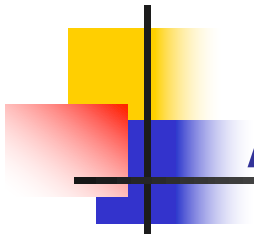
Exemple de connexion

- Les client d'un service envoient une requête CONNECT TSAP (port TCP) du service
  - S'il n'y a pas de serveur qui les attend, ils font une connexion au serveur de processus (a)
- Le serveur de processus active le serveur demandé et lui transmet la connexion avec le client demandeur (b)
  - Le nouveau serveur exécute la commande demandée
- Le serveur de processus revient à l'écoute de nouvelles requêtes

Adressage : TSAP

Exemple de connexion

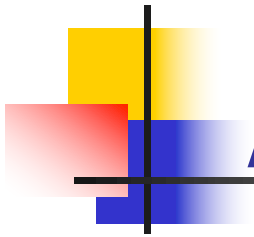




Adressage : TSAP

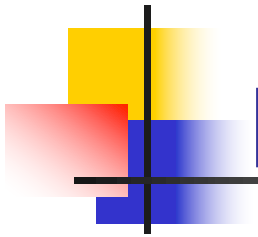
- Comment localiser la machine du serveur ?
  - Comment savoir quel NSAP l'entité transport doit utiliser pour établir une connexion réseau ?

- TSAP avec une structure hiérarchique :
  - L'adresse TSAP va partitionner l'espace adressable
    - TSAP = galaxie | étoile | planète | pays | ville | machine | port
    - TSAP = 33 1 44 27 71 26
    - TSAP = adresse IP | port



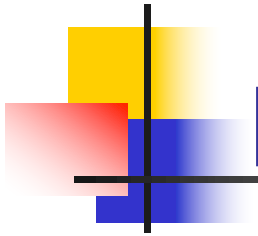
Adressage : TSAP

- Structures à plat
  - flat address space, adresse globale
  - besoin d'une mise en correspondance supplémentaire :
    - Annuaire qui donne les adresses réseau en fonction des adresses de transport
    - Dans le cas des LANs, diffusion d'un appel général demandant au destinataire de donner son adresse



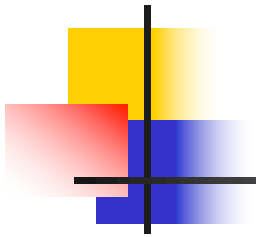
Établissement de Connexion

- Établissement de la connexion en 2 ou 3 phases
- Deux séquences d'échange de messages lors de l'établissement de la connexion peuvent être définies :
  - En deux phases ("two way handshake") :
    - service Réseau assure la séquentialité (généralement en mode connecté)
  - En trois phases ("three way handshake")
    - service Réseau n'assure pas la séquentialité (généralement en mode non connecté)



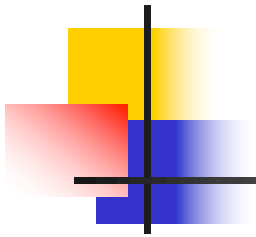
Etablissement de Connexion

- Envoi d'un CONNECTION_REQUEST TPDU
- mais possibilité de :
  - perte
  - mémorisation
  - duplication
    - peut créer deux connexions...
- Il faut traiter le problème de duplications



Etablissement de Connexion

- éviter la duplication avec :
  - génération d'un nouveau TSAP à chaque connexion
    - Le Modèle Serveur de processus est impossible à mettre en place
  - utilisation d'identificateurs uniques de connexion (état)
    - Nécessite que chaque entité transport gère une table de connexions
    - Problème en cas de panne et pertes de mémoire
  - élimination des anciens paquets



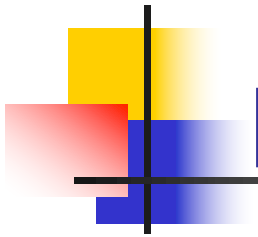
Etablissement de Connexion

Problèmes engendrés par le service Réseau

Certains messages peuvent errer dans le réseau pendant un certain temps :

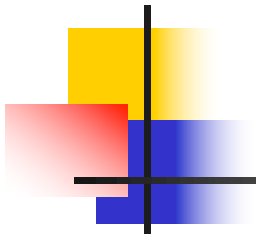
- notamment si le service Réseau est en mode non connecté
- les paquets suivent des chemins indépendants,
- le mécanisme de retransmission produit des doubles,
- certaines portions du réseau peuvent se trouver isolées du reste :
  - soit par panne de routeur ou par rupture de liaisons,
  - soit à cause de l'instabilité (vitesse de convergence) des algorithmes de routage.
- les routeurs doivent mémoriser les paquets qu'ils ne peuvent temporairement acheminer.

- Solution : Limiter la durée de résidence d'un paquet au sein du réseau
  - et adapter les mécanismes de la couche Transport en conséquence
- Exemple :
  - dans Internet, les datagrammes IP possède un champ Time To live (TTL).



Etablissement de Connexion

- élimination des anciens paquets
  - Limitation de vie des paquets
    - Limiter la taille du sous-réseau : évite que les paquets tournent en rond et les risques de congestion sur le chemin le plus long (connu)
    - Affecter un compteur de nombre de nœuds traversés à chaque paquet : incrémenter ce compteur à chaque franchissement d'un nœud, et élimination des paquets dont le compteur dépasse une certaine limite (MAX)
    - Estampiller (date et heure) chaque paquet : les routeurs éliminent ceux dont l'âgé a dépassé une certaine valeur
      - Nécessité de synchronisation des horloge

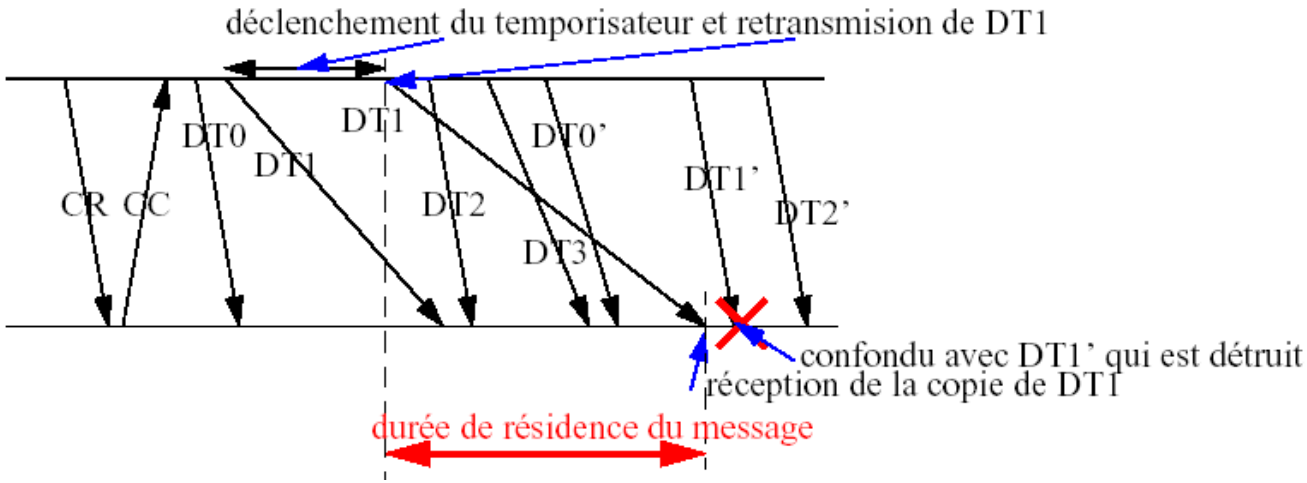


Numéros de Séquence

- Principe de base : deux TPDU avec le même numéro de séquence ne peuvent se rencontrer
  - utilisation de l'horloge pour l'initialisation
  - progression limité : existence d'une zone interdite
  - problème de la réutilisation des numéros de séquence
    - attente d'un délai T assurant la fin des connexions précédentes

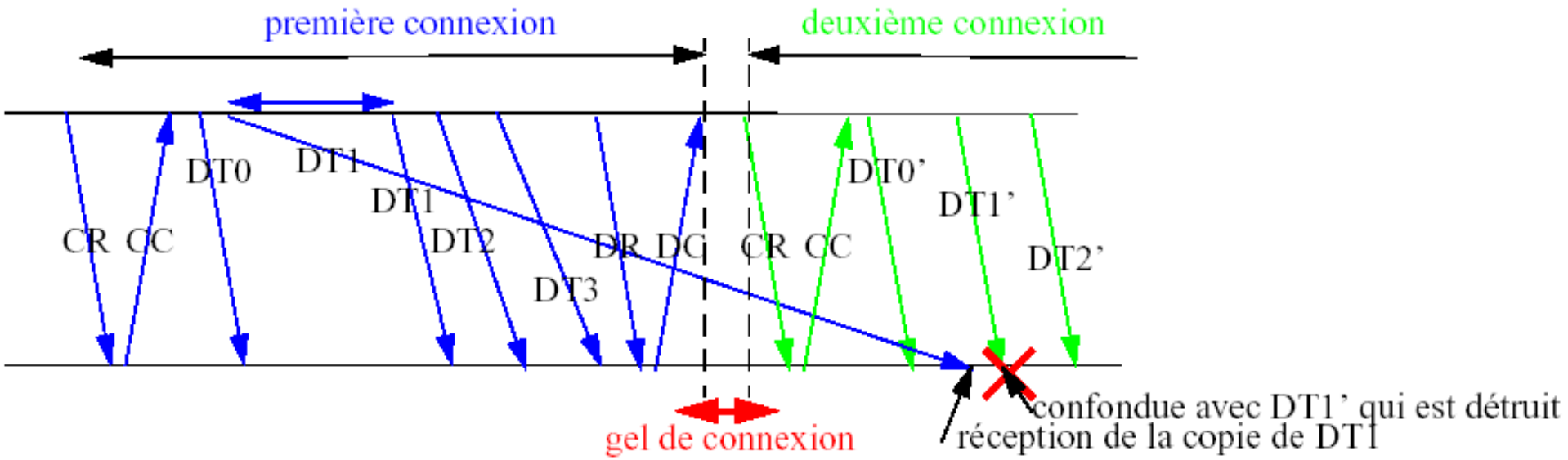
Numéros de Séquence

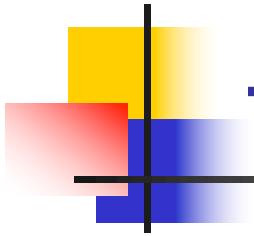
- Au sein d'une même connexion
  - Les messages sont identifiés par une numérotation cyclique !
  - La durée d'utilisation du cycle doit être supérieure à la durée maximale de vie des messages au sein du réseau.
- Une solution à ce problème peut être apportée grâce à un plus grand cycle de numérotation :
  - modulo 27 --> modulo 215 !
  - ou une durée de résidence plus faible.
- Autre propriété : durée du temporisateur de retransmission > durée de résidence



Numéros de Séquence

- Confusion entre connexions
  - Réutilisation de l'identificateur de connexion après une durée de garde supérieure à la durée maximale de vie des messages au sein du réseau.
  - La référence d'une connexion libérée est gelée pendant cette durée.
- Propriété : durée du gel de connexion > durée de résidence des messages



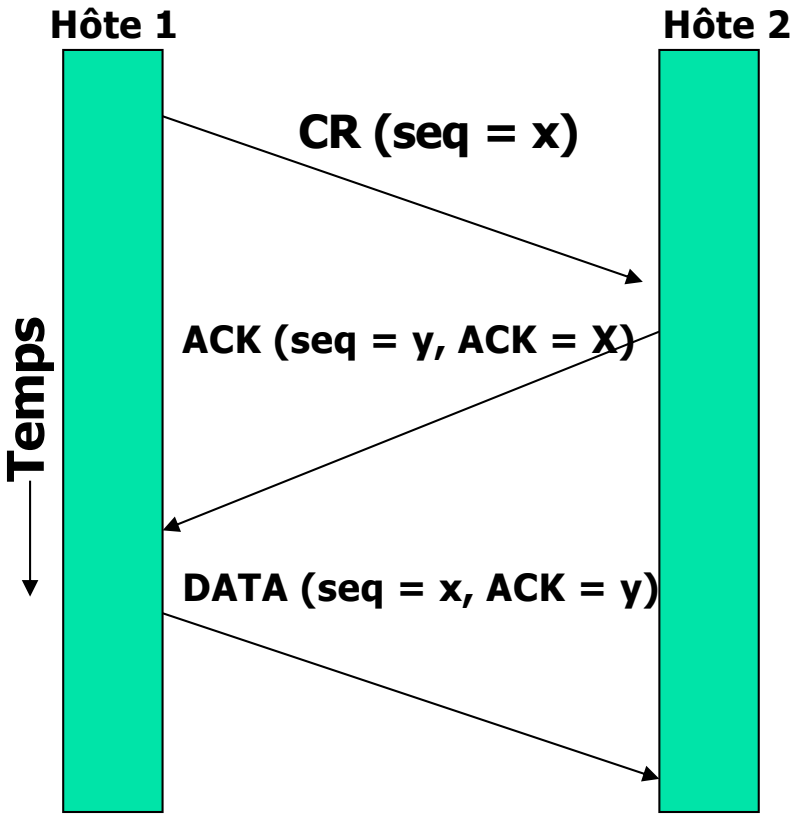


Three Way Handshake

- Etablissement d'une connexion bidirectionnelle avec numéros de séquences indépendants

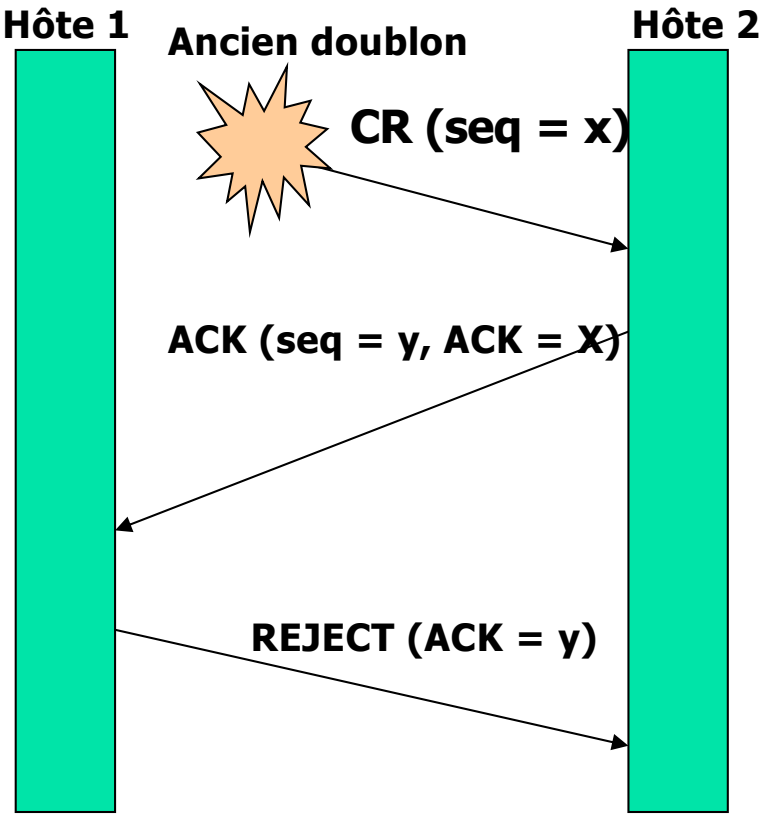
- Utilisation d'un échange de trois TPDU :
  - CONNECTION_REQUEST (CR)
    - initiateur envoi son numéro de séquence vers le destination
  - CONNECTION_ACCEPTED (ACC)
    - destination acquitte et envoi le sien vers initiateur
  - DATA
    - initiateur acquitte avec les premières données vers destination

Établissement de connexion à 3 étapes



a)

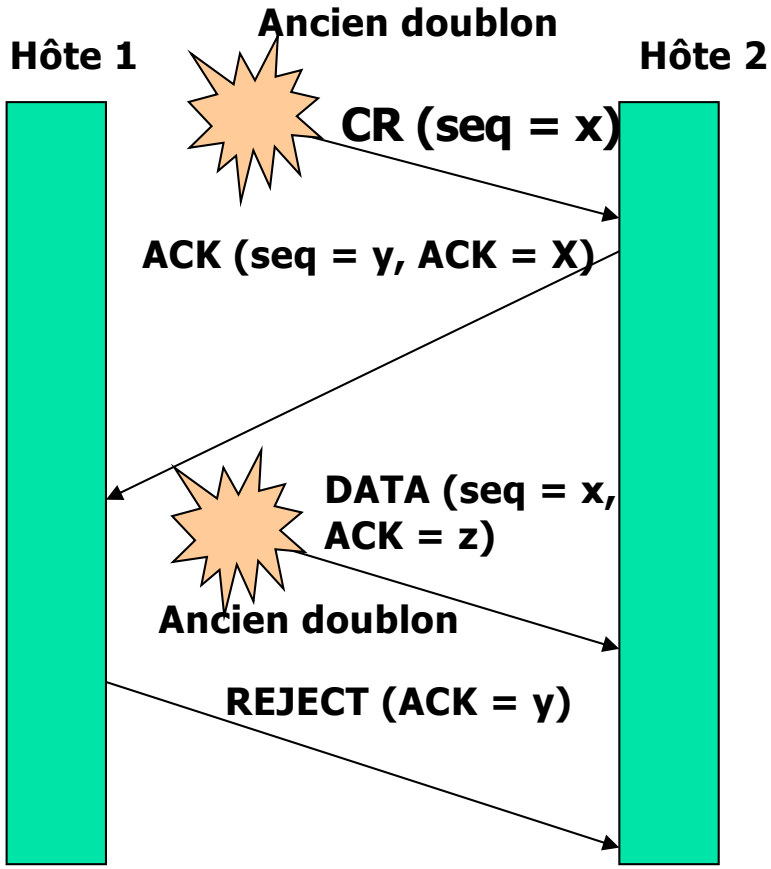
Fonctionnement normal



b)

Ancien CR dupliqué resurgissant

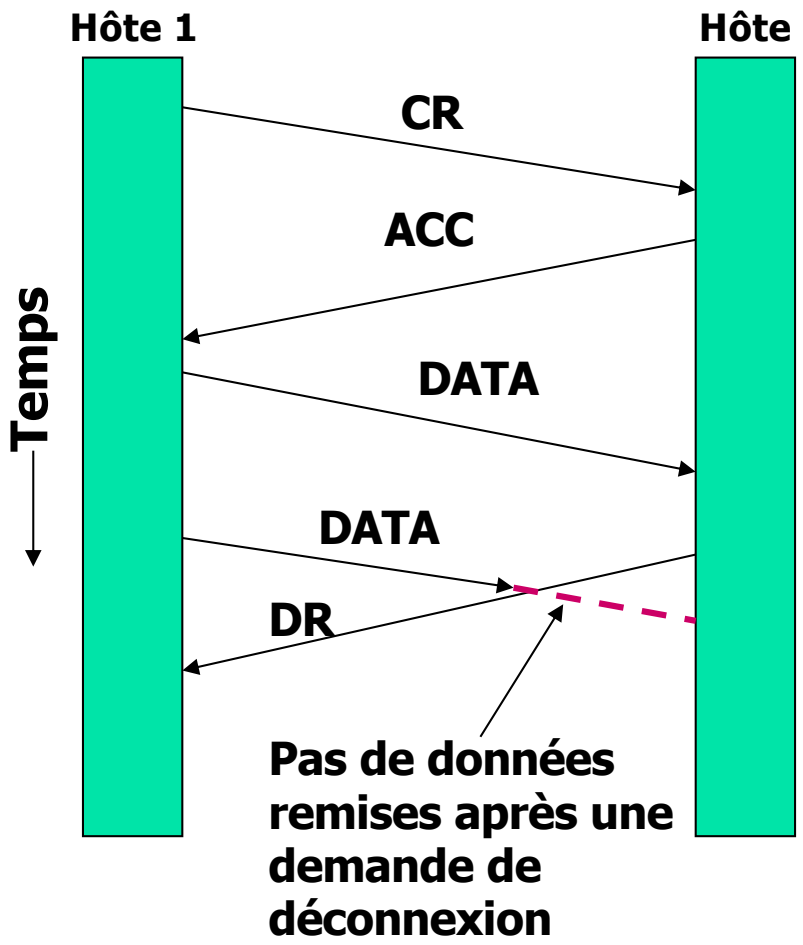
Établissement de connexion à 3 étapes



CR et ACK dupliqués

c)

Terminaison de Connexion



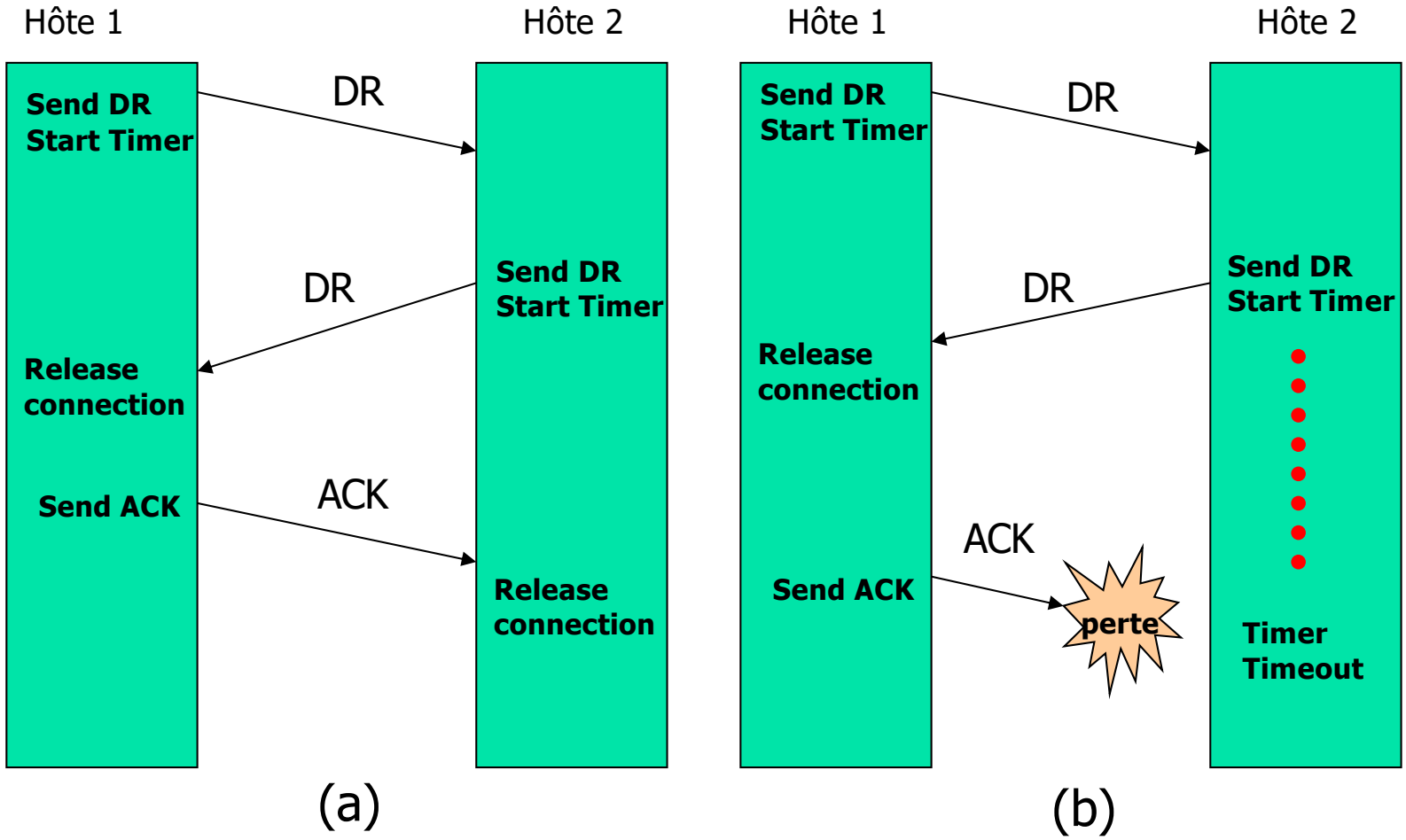
- Deux types :
- asymétrique
  - fin de connexion dès qu'une des extrémités le demande
  - pertes possibles (schéma)
- symétrique
  - terminaison indépendantes des deux directions
  - fin de connexion lorsque chaque extrémité à terminé
    - Two army problem

Problème de synchronisation

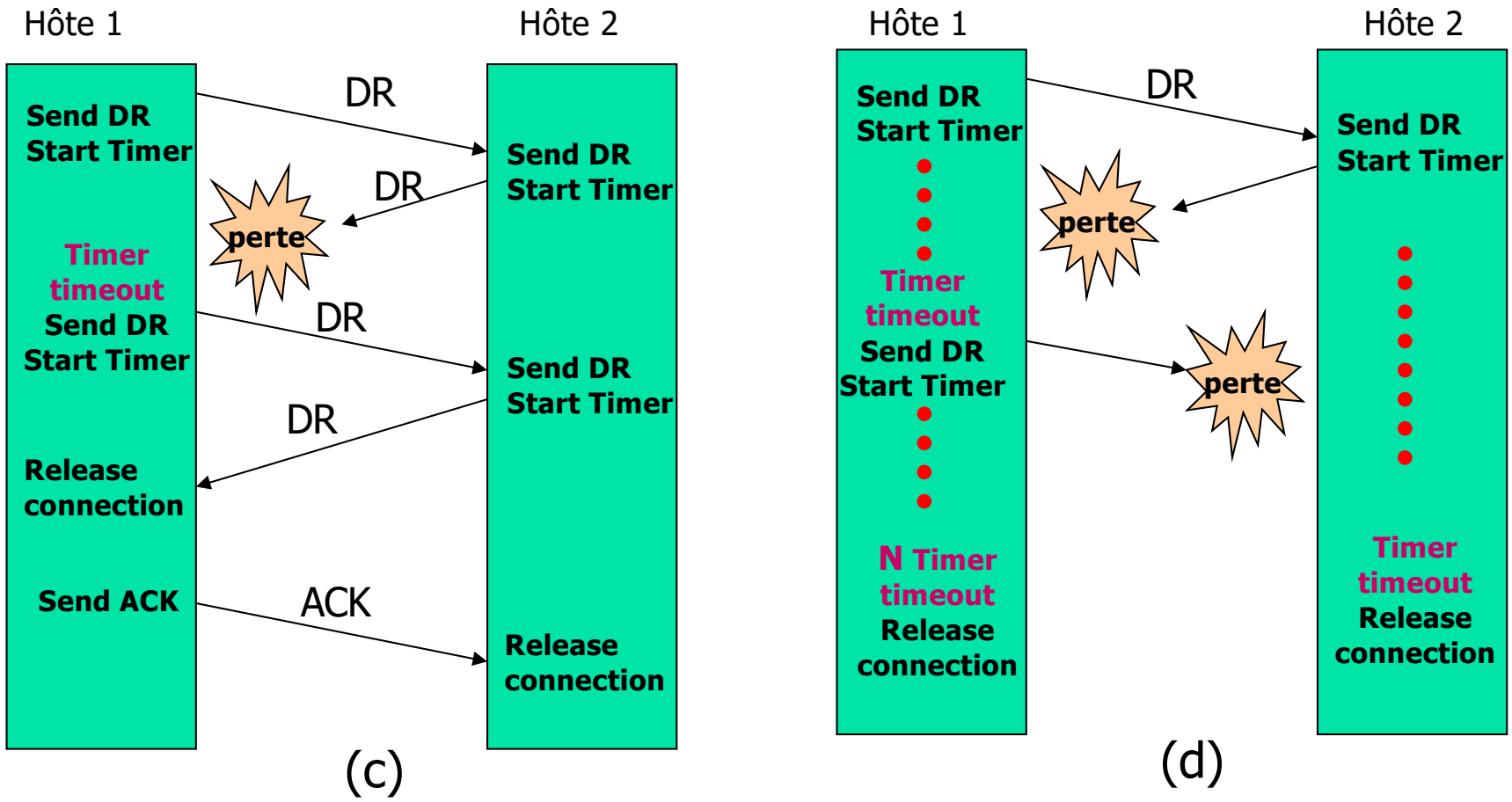


- Problème de synchronisation
  - les deux armées bleues doivent attaquer conjointement :
    - quel protocole de communication fiable ?
      - un messager peut être intercepté
      - les commandements doivent être acquittés
      - les acquittement aussi ... pas de solution !
    - Même mécanisme pour les déconnexions
      - introduction de temporisation

Exemple de Terminaison (1)

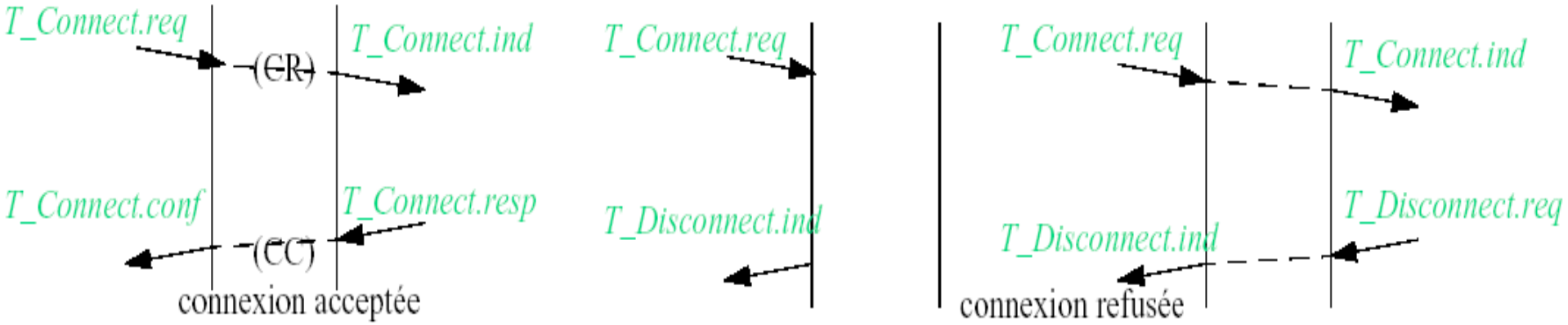


Exemple de Terminaison (2)



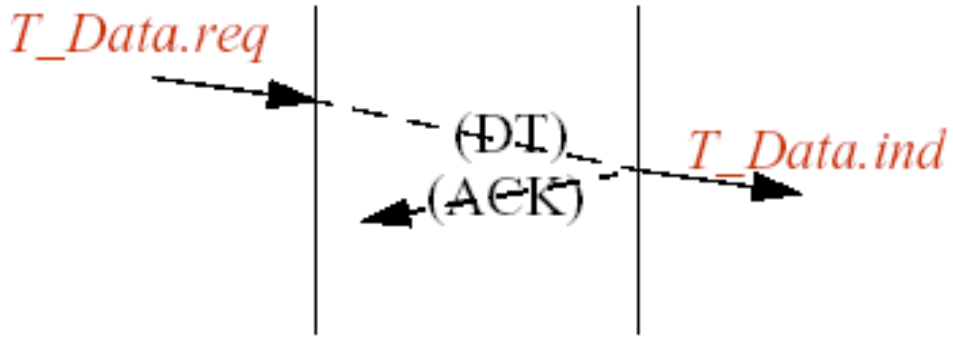
Établissement de connexion

- **En mode connecté**
  - Établissement de connexion avec négociation de la QoS à l'aide de quatre primitives T_CONNECT : T_Connect.req(adresse destination, adresse source, option de données exprès, qualité de service, données utilisateur), T_Connect.ind(idem), T_Connect.resp(adresse de réponse, option de données exprès, qualité de service, données utilisateur) et T_Connect.conf(idem).



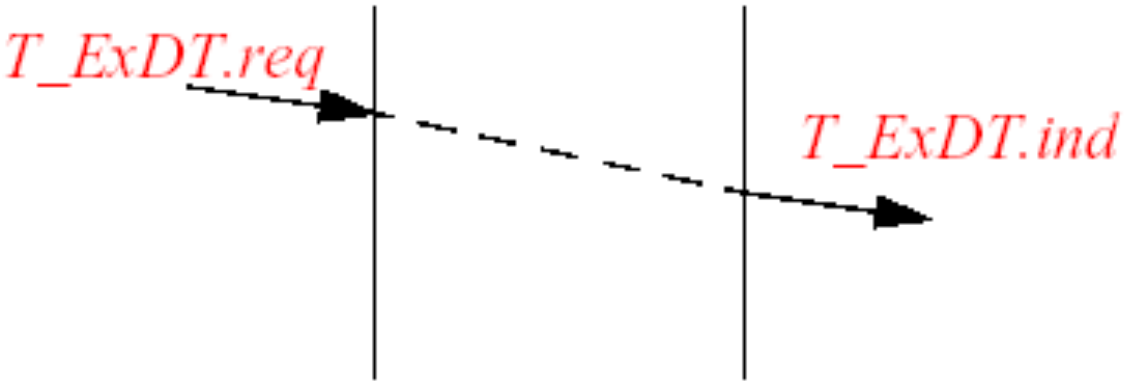
Transfert de données

- Transfert de données (“normales”) avec contrôle de flux sur une connexion de Transport avec les primitives *T_D*ata : *T_D*ata*.req(données utilisateur)* et *T_D*ata*.ind(idem)*



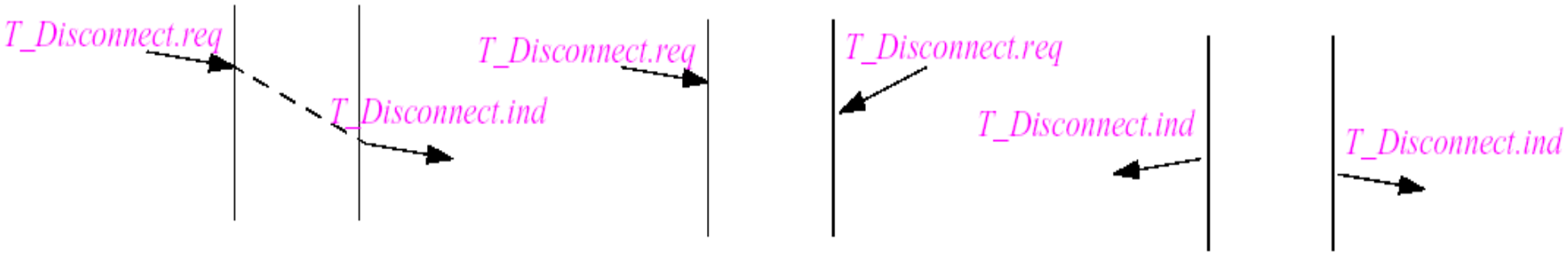
Transfert de données

- Transfert de données exprès (optionnel) avec les deux primitives *T_Expedited_DATA* :*T_Expedited_Data. req(données utilisateur)* et *T_Expedited_Data.ind(idem)*.



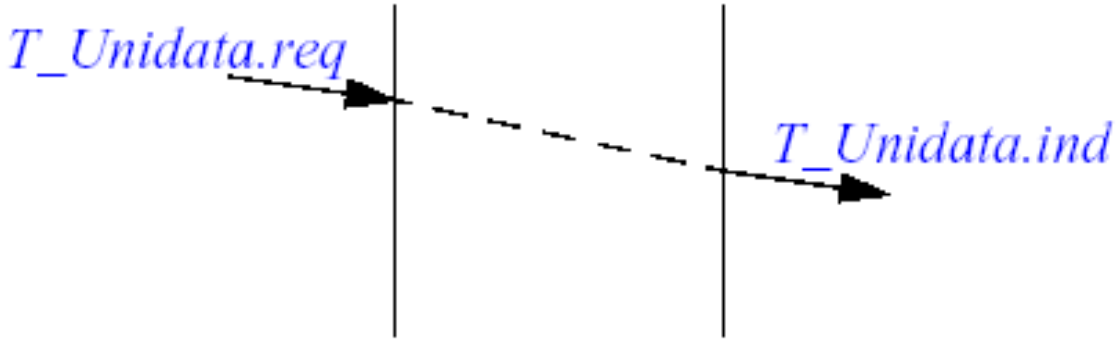
Libération de la connexion

- Libération de la connexion avec les primitives *T_Disconnect* : *T_Disconnect.req(données utilisateur)* et *T_Disconnect.ind(cause de la déconnexion, données utilisateur)*.



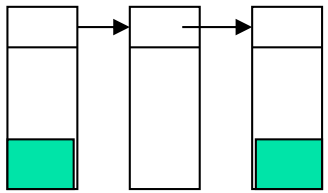
Transfert de données

- En mode non connecté:
  - L'utilisateur du service Transport ne dispose que de deux primitives T_Unidata liées au transfert de données : T_Unidata.req(adresse destination, adresse source, qualité de service, données utilisateur) et T_Unidata.ind(idem).
- Remarque : Ces deux primitives véhiculent tous les paramètres nécessaires à leur transmission (adressage, QoS,...) alors que les primitives T-Data du mode connecté n'en ont pas besoin.



5. Contrôle de flux et mémorisation

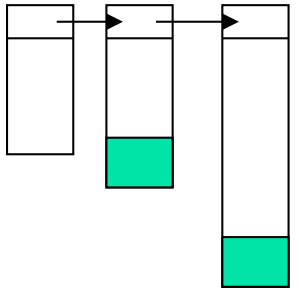
- Gestion d'une connexion en service
- Si la couche réseau est non fiable
  - nécessite de mémoriser les TPDU expédiées
- Comment dimensionner les buffers ?
  - Si les TPDU sont de taille similaire
    - gestion d'un ensemble de buffers de taille fixe (a)



(a)

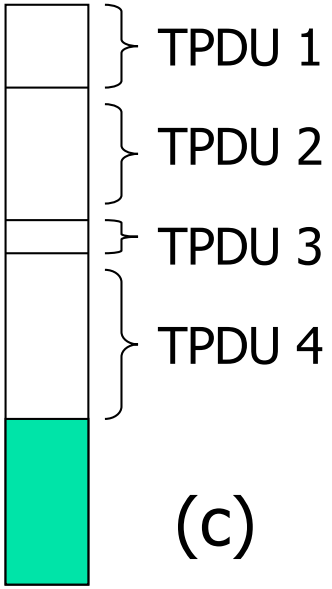
Contrôle de flux et mémorisation

- Si les TPDU sont de taille variable

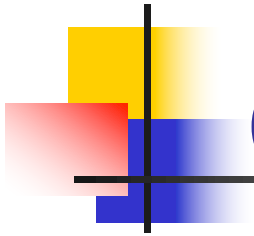


(b)

  - Ensemble de buffers de taille variable (b)
  - Buffer circulaire (c)

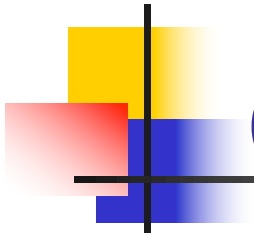


(c)



Contrôle de flux et mémorisation

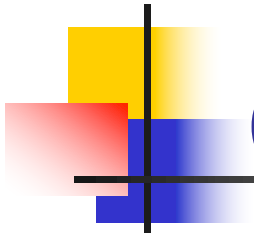
- Utilisation des acquittements et de la quantité de donnée admissible à la destination pour définir de buffers
  - Contrôle de flux
- Compromis entre mémorisation à la source ou à la destination dépend du trafic
  - Connexions multiples mais peu chargés (terminaux interactifs) ➔ ne pas mémoriser mais acquitter dynamiquement des deux extrémités



Contrôle de flux et mémorisation

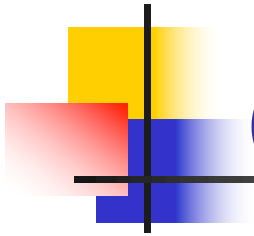
- Pour le transfert des fichiers ou les trafics importants ➔ il est préférable que les destinataire dispose d'un ensemble des tampons dédiés, afin de permettre une transmission au débit maximum
  - Pour un trafic important émis par à-coups, il est conseillé de mémoriser chez l'expéditeur
  - Alors que pour un trafic important et relativement continu, il est préférable de mémoriser chez le destinataire

- Ajuster dynamiquement la localisation et la taille des tampons alloués en fonction de la nature du trafic



Contrôle de flux et mémorisation

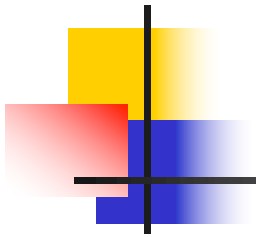
- Le protocole de transport doit permettre à un hôte de demander et de gérer des tampons au destinataire
- Ces bufferts peuvent être alloués spécifiquement à une connexion, ou collectivement pour toutes les connexions existant entre ces deux hôtes



Contrôle de flux et mémorisation

- Utilisation d'un système de fenêtrage
  - Façon de gérer l'allocation dynamique des tampons
  - Gestion de fenêtre de taille variable
    - Au départ l'expéditeur demande un certain nombre de tampons en fonction des besoins qu'il perçoit
    - Le destinataire lui en alloue autant qu'il peut
    - A chaque transfert d'un TPDU, l'émetteur doit décrémenter son allocation et suspendre la transmission lorsqu'elle devient nulle
    - Le destinataire superpose alors les acquittement et les allocations au trafic en retour (piggybacking)
  - Exemple figure suivante

A	message	B	Signification
1 →	seq = 14, buffer = 8	→	A demande 8 tampons (A peut envoyer 8 TPDU)
2 ←	ack = 14 ,buffer = 4	←	B n'accepte que les messages 0 à 3 (acceptation de 4 par B)
3 →	seq = 1, data = m1	→	A a 3 tampons libres (envoi de 1, reste 3 à A)
4 →	seq = 2, data = m2	→	A a 2 tampons libres (envoi de 1, reste 2 à A)
5 →	seq = 3, data = m3	…	Message perdu (A pense : envoi de 1, reste 1 à A)
6 ←	Ack = 2, buffer = 3	←	B acquitte les messages 1 et 2, autorise 2 à 4
7 →	seq = 4, data = m4	→	A a 1 tampon libre
8 →	seq = 5, data = m5	→	A n'a plus de tampon, il arrête
9 →	seq = 3, data = m3	→	A retransmet après expiration du timer
10 ←	Ack = 5, buffer = 0	←	Tous les messages sont acquittés mais A est bloqué
11 ←	Ack = 5, buffer = 1	←	A peut envoyer le message 6
12 ←	Ack = 5, buffer = 2	←	B a trouvé un tampon
13 →	seq = 6, data = m6	→	A a un tampon libre (envoi de 1, reste 1 à A)
14 →	seq = 7, data = m7	→	A est à nouveau bloqué (envoi de 1, reste 0 à A)
15 ←	Ack = 7, buffer = 0	←	A est toujours bloqué
16 …	Ack = 7, buffer = 0	←	Risque de blocage définitif



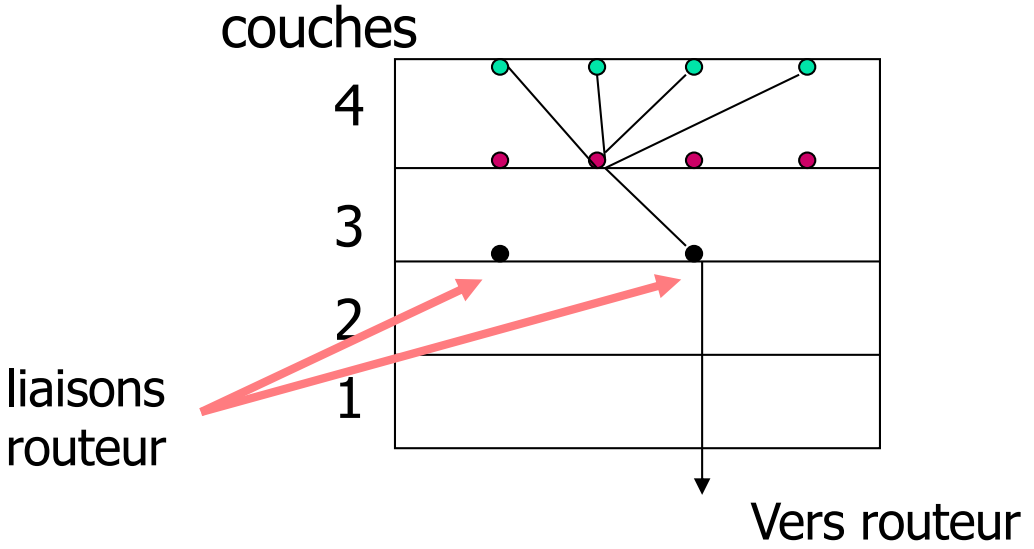
Multiplexage

- Plusieurs conversations sur une même connexion, un même circuit virtuel, une même ligne

- Au niveau de la couche transport on retrouve deux types de multiplexage :
  - Multiplexage ascendant (amont)
  - Multiplexage descendant (aval)

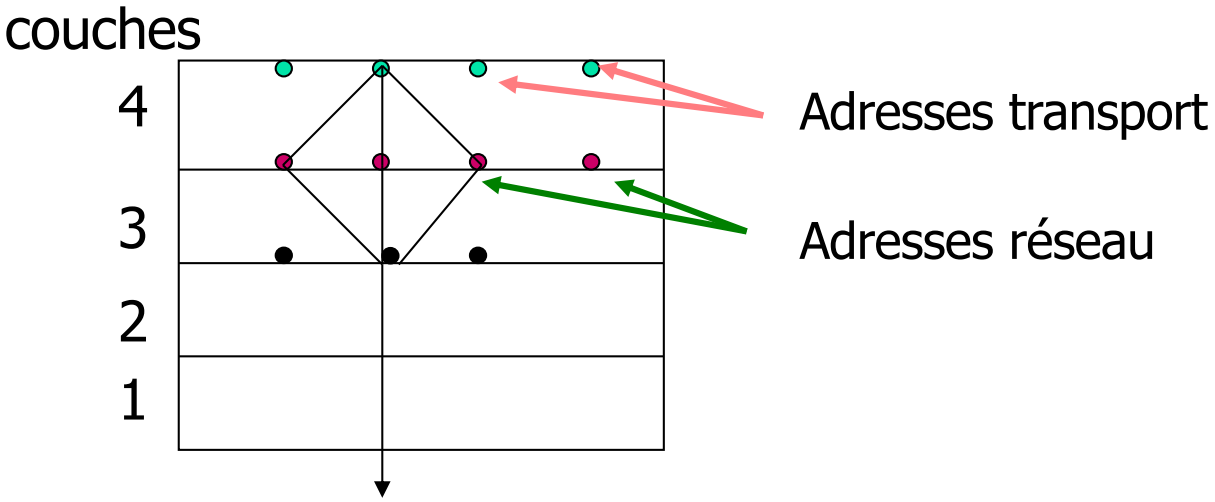
Multiplexage

- Multiplexage ascendant :
  - Multiplexage des connexions transport sur une même connexion réseau pour partager les ressources réseau (généralement coûteuses)

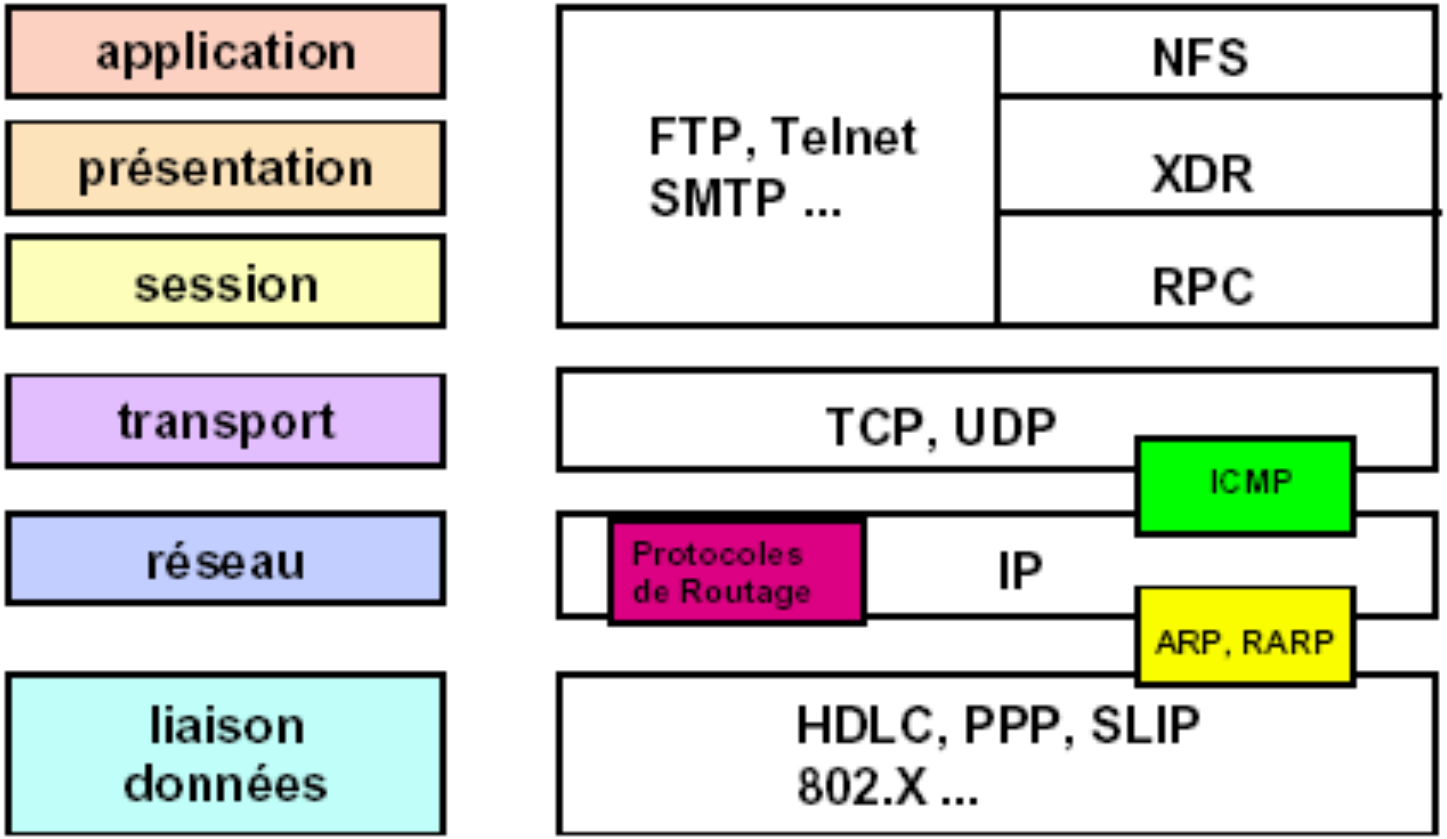


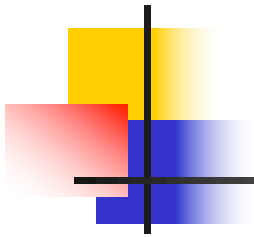
Multiplexage

- Multiplexage descendant :
  - La couche transport ouvre plusieurs connexions de réseau et distribue le trafic entre elles de manière équitable
  - possible pour réaliser du partage de charge...



6. Pile Protocole IP

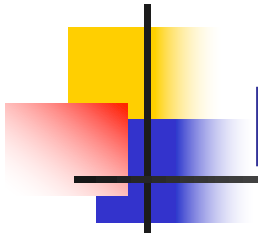




Echanges d'informations

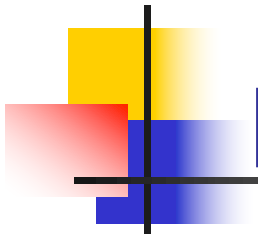
Comment ça marche ?

- **Transformation du message initial**
- **Fragmentation par la couche transport**
  - Segments
- **+ En-tête IP (@source,@destination)**
  - Datagrammes
- **Envoi sur le support physique de transport**
  - Trames, cellules



Echanges d'informations

- **Mode connecté : TCP**
  - **=> type communication téléphonique**
    - phase de connexion / déconnexion
    - phase de transfert des données
- **Mode non connecté : UDP**
  - **=> type envoi d'une lettre**
    - pas de phase préalable à l'envoi des données



Protocoles Internet

- **Principalement deux services (et protocoles) :**
  - **TCP (*Transmission Control Protocol*)**
    - protocole **orienté connexion fiable** avec contrÙle de **flux** et de **congestion**
  - **UDP (*User Data Protocol*)**
    - protocole offrant le même service qu'IP (**datagramme non fiable**) au niveau transport

Mécanisme d'encapsulation

Couche application

entête applicative

Couche transport (TCP)

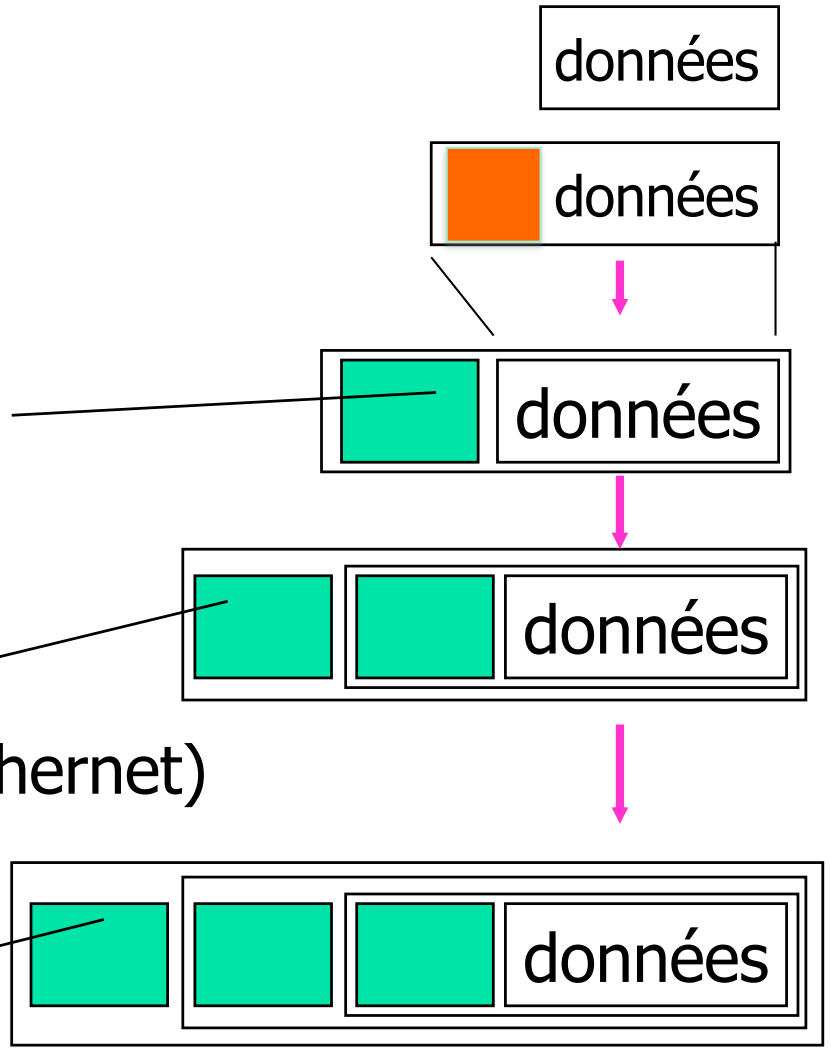
entête TCP (~20 octets)

Couche réseau (IP)

entête IP (~20 octets)

Couche liaison (p.ex. Ethernet)

entête Ethernet (~18 octets)



Données utilisateur

message

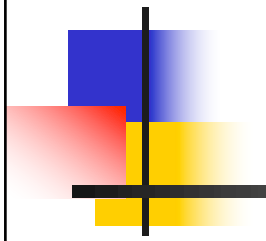
segment

paquet

trame

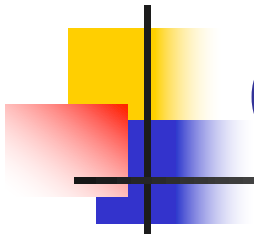
UNIVERSITÉ SORBONNE PARIS NORD

Institut GALILÉE
Université Sorbonne Paris Nord



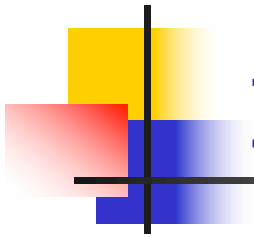
☞ Séance 6

Couche Transport (2/2)



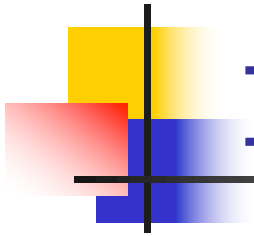
Couche Transport (2/2)

1. Protocole TCP
   - Description générale
   - L'en-tête du segment TCP
   - Gestion d'une connexion TCP
   - Politique de transmission TCP
2. Protocole UDP
   - Description générale
   - L'en-tête du segment UDP



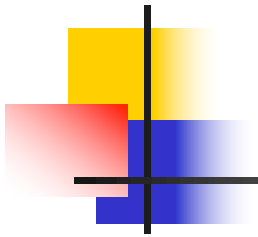
1. Introduction à TCP

- Le protocole TCP est un protocole de communication qui fonctionne au dessus d'IP afin d'assurer :
  - la transmission de données de taille quelconque en mode orienté connexion ;
  - la restitution à l'application des données dans l'ordre et sans doublon ;
  - le multiplexage de plusieurs communications sur une seule connexion ;
  - le contrôle de flux ;
  - les retransmissions en cas de perte ou d'erreur.



Introduction à TCP

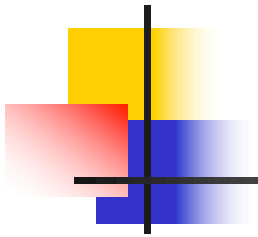
- Ce protocole a été amélioré au cours des années afin d'inclure de très nombreuses fonctionnalités pour permettre d'augmenter ses possibilités et de le rendre aussi efficace que possible
- Il inclut, entre autres :
  - une procédure de connexion dite « à trois mains »
  - un contrôle de flux
  - une gestion dynamique de fenêtre
  - un procédure de déconnexion



Introduction à TCP

- TCP est un protocole qui procure un service de flux d'octets orienté connexion et fiable
- Les données transmises par TCP sont encapsulées dans des datagrammes IP en y fixant la valeur du protocole à 6

- ➔ Références TCP :
  - RFC 793 (+ RFC 1122 et RFC 1323)



Introduction à TCP

- Si les données à transmettre pour une application sont trop volumineuses,
  - les données sont fractionnées en fragments dont la taille est jugée optimale par TCP

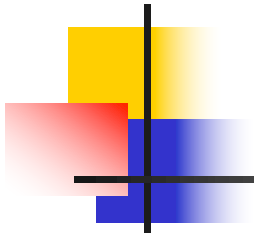
- A l'inverse, TCP peut regrouper des données d'une application pour ne former qu'un seul datagramme de taille convenable de manière à ne pas charger inutilement le réseau.

- Cette unité d'information émise est appelée ***segment***

Fonctionnalités de TCP

- **Type de transmission**
  - Bout-en-bout
  - Orienté connexion
  - Point-à-point
  - Bidirectionnel
  - Orienté flot de donnés

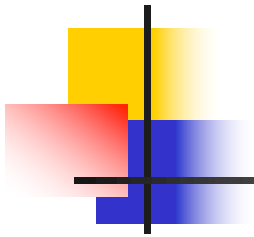
- **Fiabilité et Robustesse**
  - contrôle d'erreur
  - ordonnancement
- **Adaptation et Dynamisme**
  - contrôle de flux
  - contrôle de congestion
- **Multiplexage et Ports**



Fonctionnalités de TCP

TCP : Type de Transmission

- Transmission de bout-en-bout
  - *end-to-end Transport Service*
  - abstraction du réseau
- Mode orienté connexion
  - les applications dialoguant à travers TCP sont considérées l'une comme un ***serveur***, l'autre comme un ***client***,
  - elles doivent établir une connexion avant de pouvoir dialoguer



Fonctionnalités de TCP

- TCP traite les données venant des couches supérieurs comme une suite d'octets
- Découpe cette suite d'octets en segments
  - Taille maximal est de 64 Koctets
- 1 segment TCP est contenu dans un datagramme IP
  - Champ du protocole du datagramme IP = 6
- Des segment sont échangés pour :
  - Ouvrir des connexions
  - Transfert de données
  - Envoyer des Ack, gérer le contrôle de flux
  - Fermer les connexions

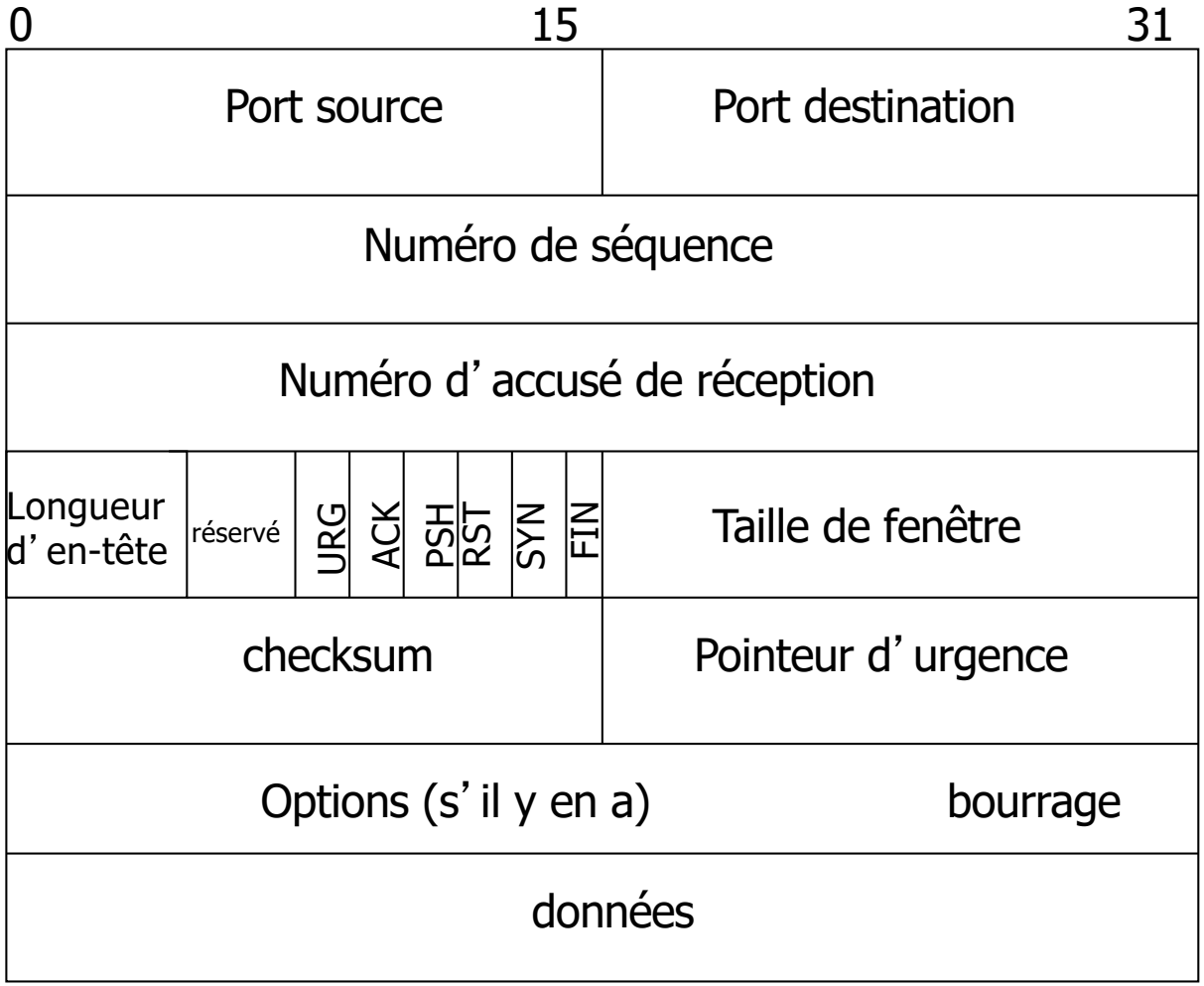
Segments TCP

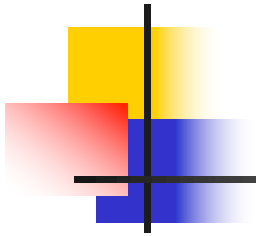
TCP : Type de Transmission

- Échange de **segments** par les entités TCP
  - En-tête de 20 octets (+ options) = IP
  - Zéro ou plus octets de données
- Taille des segments décidés par TCP
  - Taille max = max (65535o avec en-tête TCP, MTU)
    - Taille max = 65535 octets – 20 octets (en-tête IP) – 20 octets (en-tête TCP) = 65515 octets
  - un segment doit tenir dans la MTU
    - pour Internet (IPv4) la MTU mini est 556 octets (*payload TCP* de 536 octets)
    - charge utile par défaut d'un segment TCP)= 536 octets
    - MTU 556 octets (IPv4) – 20 octets (en-tête IP)

Format d'un segment TCP

- Format d'un segment TCP qui sert aux trois fonctionnalités de TCP :
  - établir une connexion,
  - transférer des données et
  - libérer une connexion

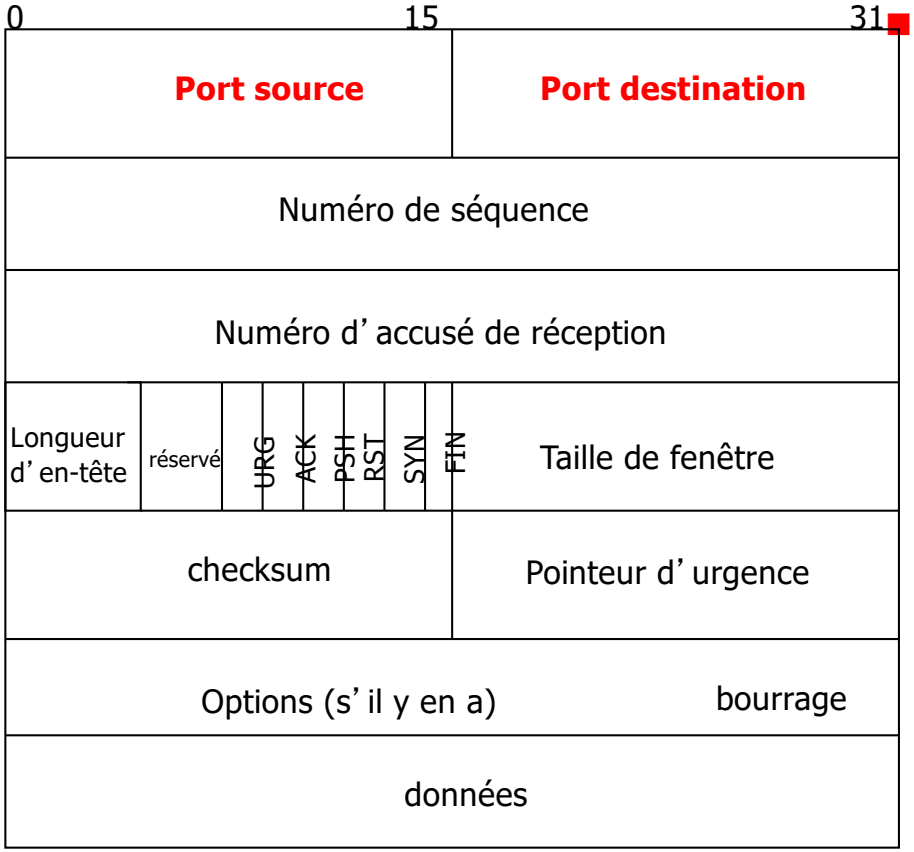




Format d'un segment TCP

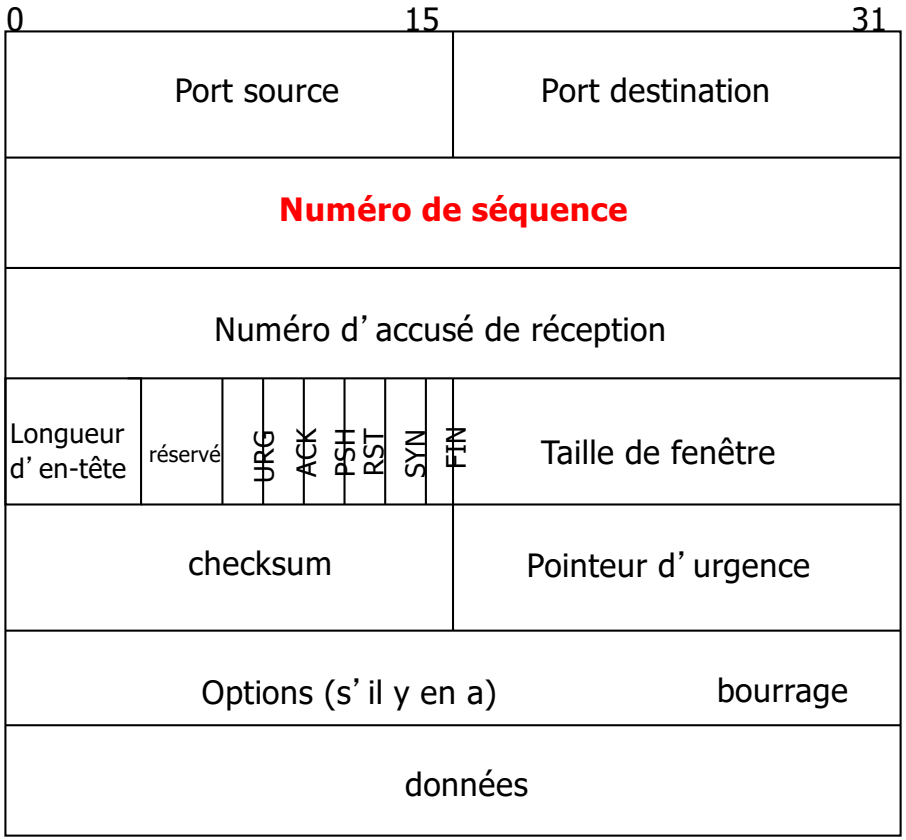
- TCP: Taille minimale est de 20 octets
  - L'en-tête, sans option, d'un segment TCP a une taille totale de 20 octets
- L'entête minimale TCP/IP = 40 octets
  - 20 octets de l'en-tête TCP + 20 octets de l'en-tête IP

Format d'un segment TCP

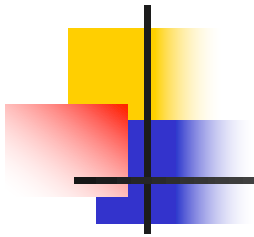


- Le *port source (16 bits)* et le *port destination (16 bits)* identifient les applications émettrice et réceptrice
  - En les associant avec les numéros IP source et destination du datagramme IP qui transporte un segment TCP on identifie de manière unique chaque connexion

Format d'un segment TCP



- Le *numéro de séquence (32 bits)* donne la position du segment dans le flux de données envoyées par l'émetteur
- c'est-à-dire la place dans ce flux du premier octet de données transmis dans ce segment

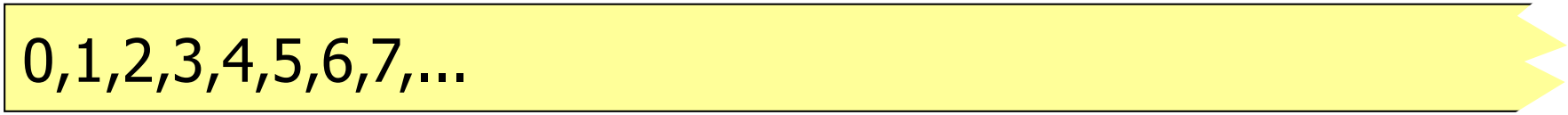


Numérotation des segments

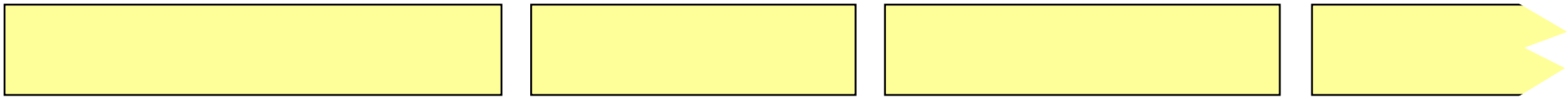
- Numéros de séquence indépendant dans chaque direction
  - associés à chaque octet
    - indique le numéro du premier octet transmis
  - mots de 32 bits
    - bouclage théorique en moins de 6 minutes à 100Mbps, mais en pratique beaucoup plus long !
  - utilisés pour les acquittements
    - si N octets sont délivrés du paquet avec le numéro de séquence X, l'acquittement aura pour valeur X+N (soit le numéro du prochain octet à recevoir)
  - *piggybacking*
    - les deux numéros sont présent dans les mêmes paquets
    - Il est possible inclure dans l'entête de segments TCP d'une communication de A vers B (données) des informations (contrôle) réferant la communication de B vers A

Numérotation des segments

Le flôt d'octets produit par la couche applicative



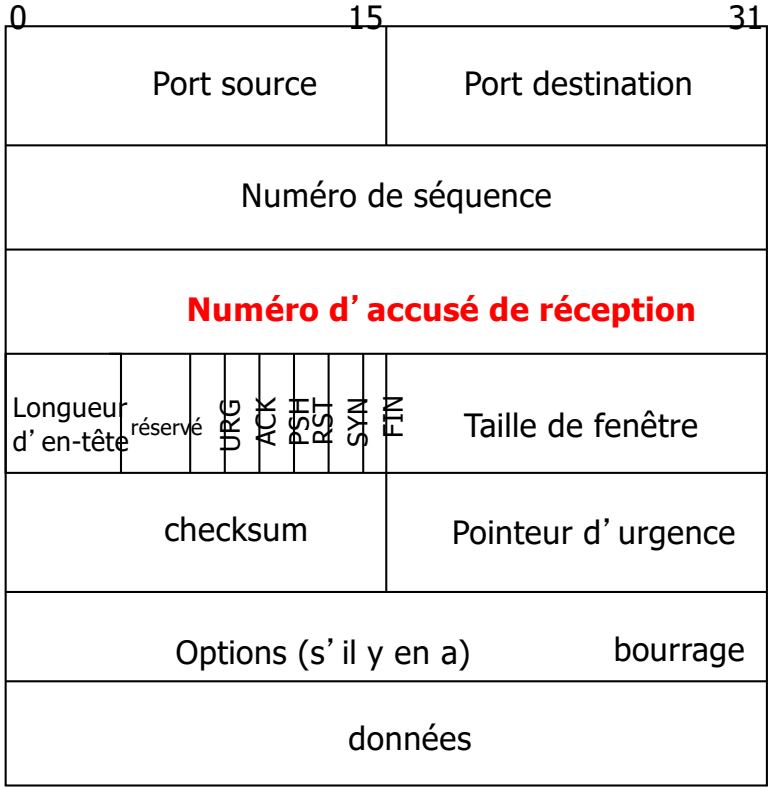
est découpé en segments pour transmission



dont les numéros de séquence ne sont rien d'autre que l'indice du premier octet

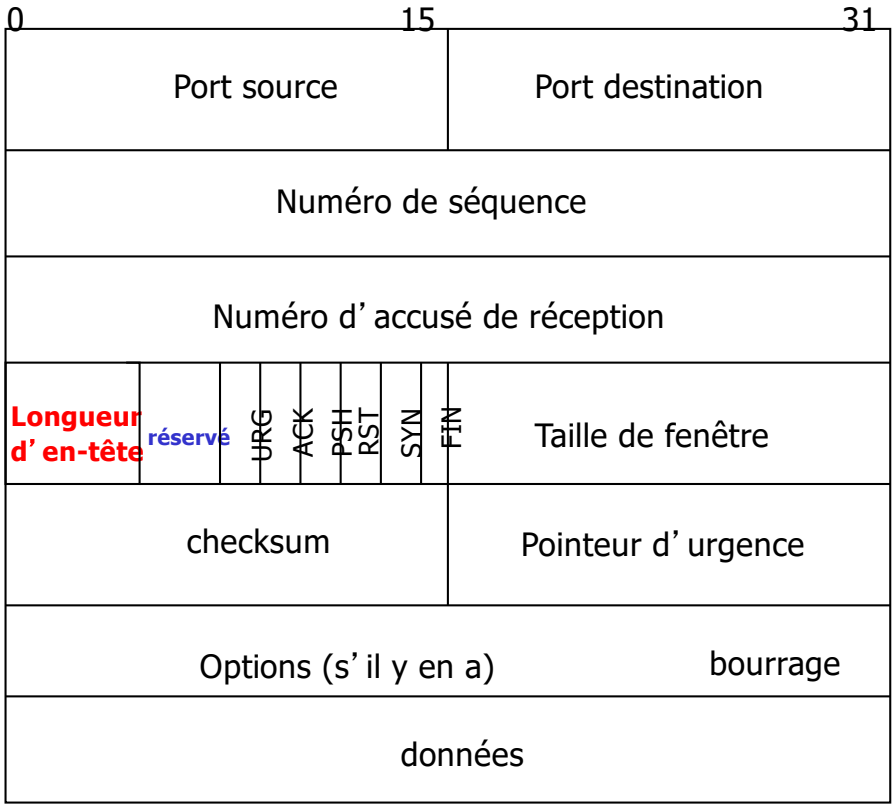


Format d'un segment TCP

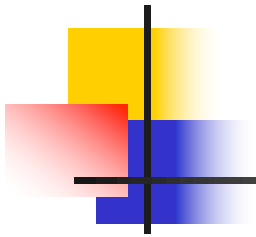


- Le *numéro d'accusé de réception (32 bits)*
  - contient en fait le numéro de séquence suivant que le récepteur s'attend à recevoir; c'est-à-dire le numéro de séquence du dernier octet reçu avec succès plus 1.
- De manière précise, TCP n'acquitte pas un à un chaque segment qu'il reçoit, mais acquitte l'ensemble du flot de données jusqu'à l'octet k-1 en envoyant un acquittement de valeur k.

Format d'un segment TCP



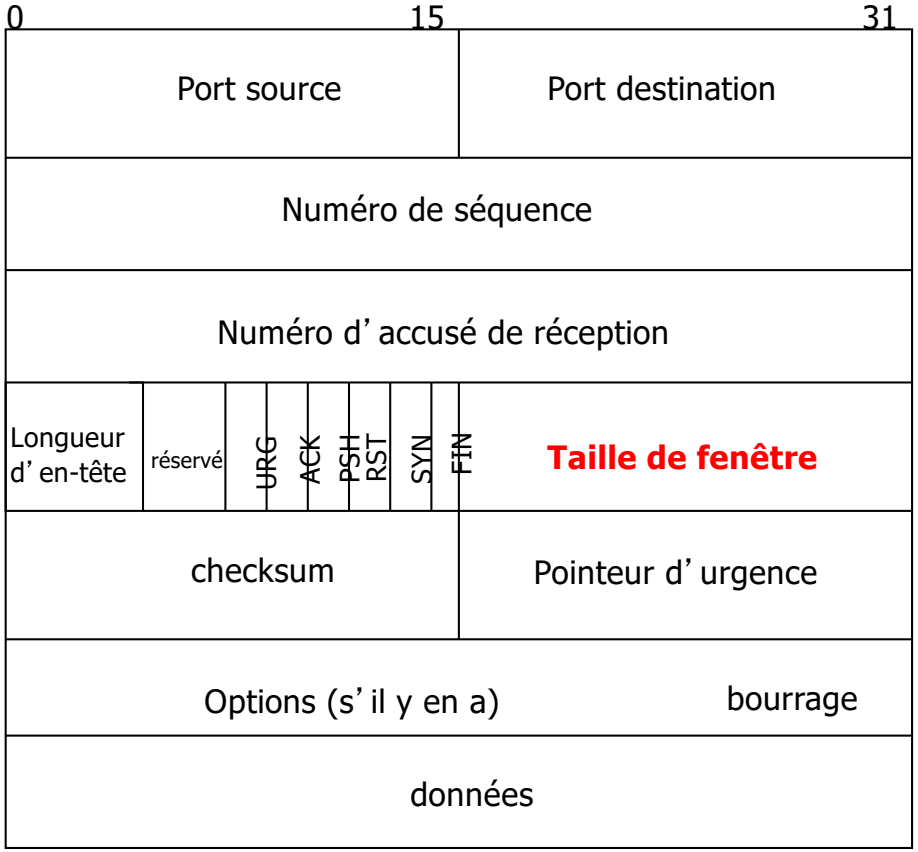
- La *longueur d'en-tête (4 bits)*
  - contient sur 4 bits la taille de l'en-tête, y compris les options présentes, codée en multiple de 4 octets.
    - Donnée en multiple de 32 bits, souvent = 5
  - Ainsi une en-tête peut avoir une taille variant de 20 octets (aucune option) à 60 octets (maximum d'options).
- Le champ ***réservé*** comporte 6 bits réservés à un usage ultérieur.



Format d'un segment TCP

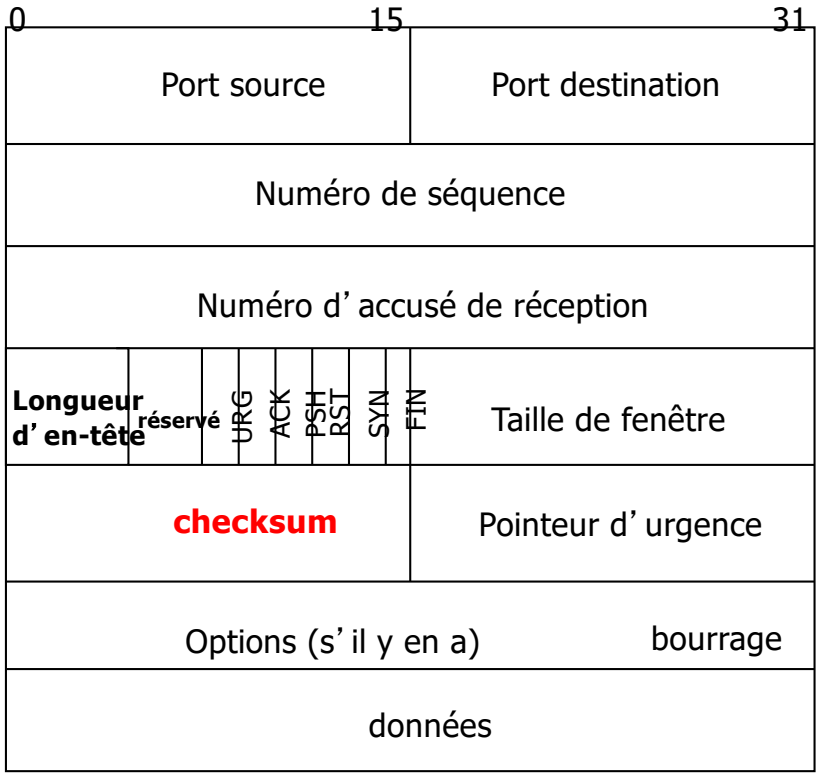
- Les 6 champs *bits de code (6 x 1 bit)* qui suivent permettent de spécifier le rôle et le contenu du segment TCP pour pouvoir interpréter correctement certains champs de l'en-tête. La signification de chaque bit, quand il est fixé à 1 est la suivante
- *URG*, le *pointeur de données urgentes* est valide.
- *ACK*, le champ *d'accusé de réception* est valide.
- *PSH*, ce segment requiert un *push*.
- *RST*, réinitialiser la connexion.
- *SYN*, synchroniser les numéros de séquence pour initialiser une connexion.
- *FIN*, l'émetteur a atteint la fin de son flot de données

Format d'un segment TCP

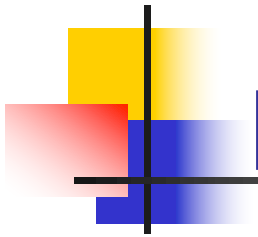


- La *taille de fenêtre (16 bits)*
  - est un champ de 16 bits qui sert au contrôle de flux selon la méthode de la fenêtre glissante.
  - Il indique le nombre d'octets (moins de 65535) que le récepteur est prêt à accepter.
  - Ainsi l'émetteur augmente ou diminue son flux de données en fonction de la valeur de cette fenêtre qu'il reçoit.

Format d'un segment TCP



- Le *checksum (16 bits)*
  - est un total de contrôle sur 16 bits utilisé pour vérifier la validité de l'en-tête et des données transmises.
  - Il est obligatoirement calculé par l'émetteur et vérifié par le récepteur.
  - Le calcul utilise une pseudo-entête analogue à celle d'UDP



Format d'un segment TCP

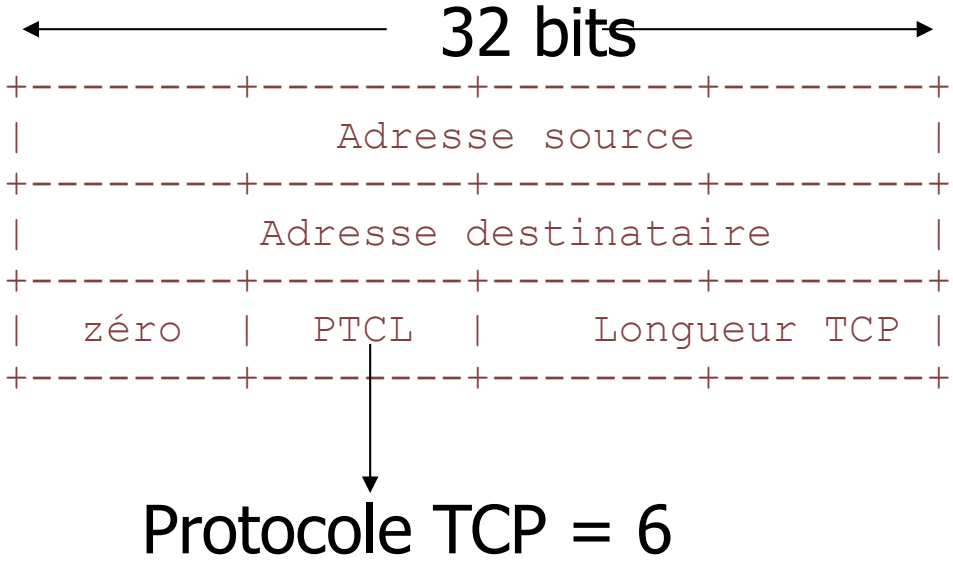
checksum

- Le Checksum est constitué en calculant le complément à 1 sur 16 bits de la somme des compléments à 1 des octets de l'en-tête et des données pris deux par deux (mots de 16 bits).
- Si le message entier contient un nombre impair d'octets, un 0 est ajouté à la fin du message pour terminer le calcul du Checksum.
- Cet octet supplémentaire n'est pas transmis. Lors du calcul du Checksum, les positions des bits attribués à celui-ci sont marqués à 0.

Format d'un segment TCP

checksum

- Le Checksum couvre de plus une pseudo en-tête de 96 bits préfixée à l'en-tête TCP.

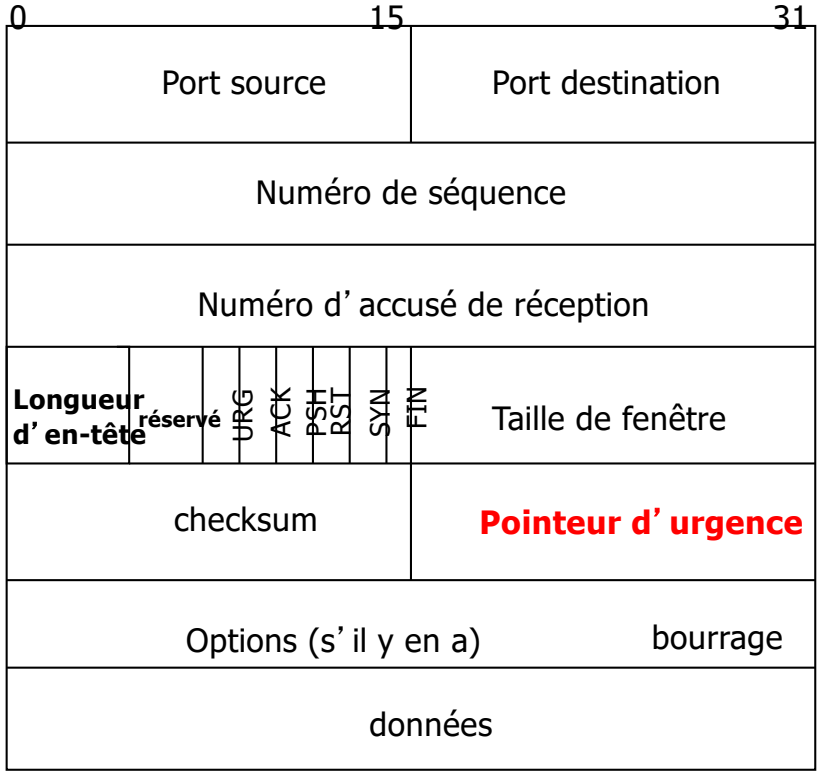


Format d'un segment TCP

checksum

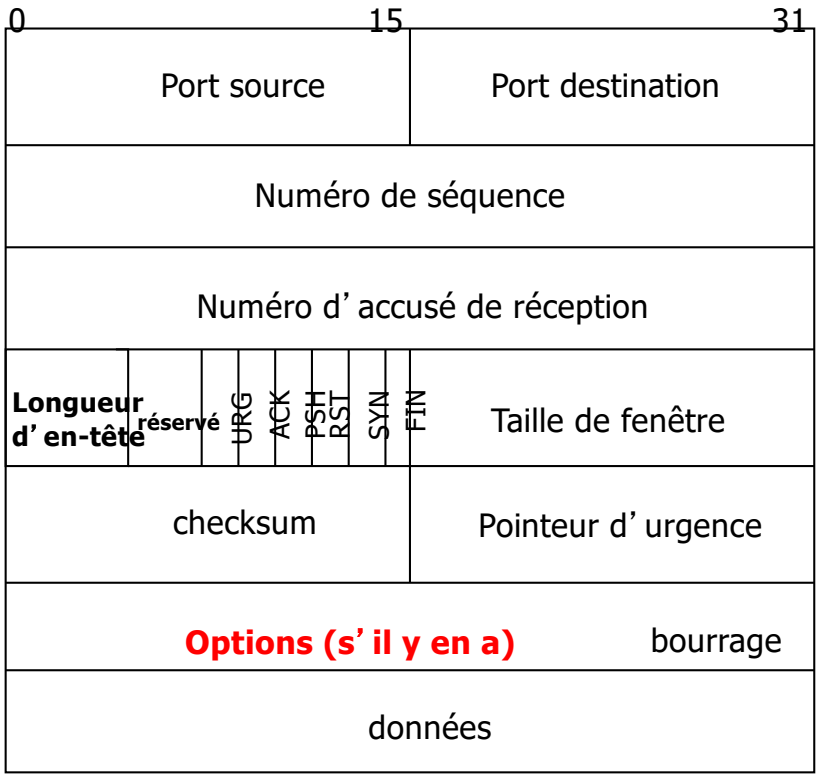
- Cette pseudo en-tête comporte les adresses Internet source et destinataires, le type de protocole et la longueur du message TCP.
- Ceci protège TCP contre les erreurs de routage.
- Cette information sera véhiculée par IP, et est donnée comme argument par l'interface TCP/Réseau lors des appels d'IP par TCP.
- La longueur TCP compte le nombre d'octets de l'en-tête TCP et des données du message, en excluant les 12 octets de la pseudo en-tête

Format d'un segment TCP

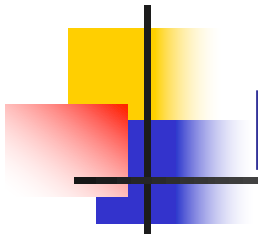


- Le *pointeur d'urgence (16 bits)*
  - est un offset positif qui, ajouté au numéro de séquence du segment, indique le numéro du dernier octet de donnée urgente.
  - Il faut également que le bit *URG* soit positionné à 1 pour indiquer des données urgentes que le récepteur TCP doit passer le plus rapidement possible à l'application associée à la connexion.

Format d'un segment TCP



- L'*option* (de taille variable) la plus couramment utilisée est celle de la taille maximale du segment TCP qu'une extrémité de la connexion souhaite recevoir.

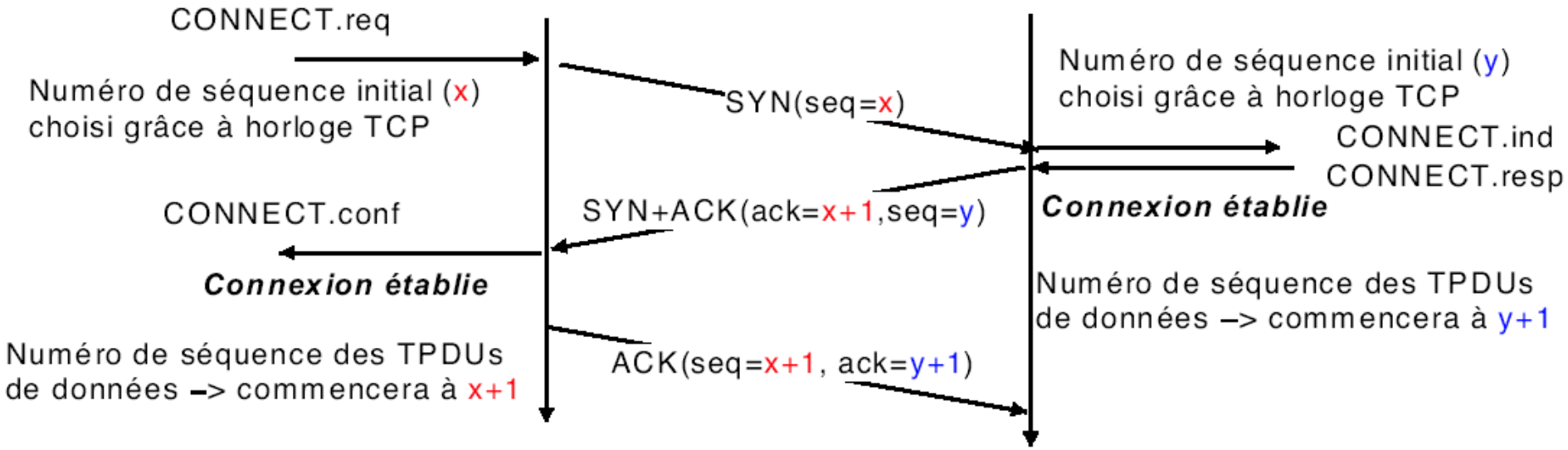


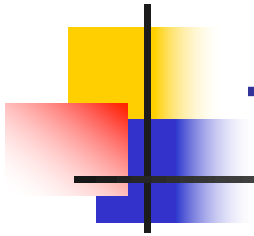
Format d'un segment TCP

- Ainsi, lors de l'établissement de la connexion il est possible d'optimiser le transfert de deux manières.
  - Sur un réseau à haut débit, il s'agit de remplir au mieux les paquets, par exemple en fixant une taille qui soit telle que le datagramme IP ait la taille du MTU du réseau.
  - Sinon, sur un réseau à petit MTU, il faut éviter d'envoyer des grands datagrammes IP qui seront fragmentés, car la fragmentation augmente la probabilité de pertes de messages

TCP : Gestion des connexions

- Remarques :
  - l'utilisation du drapeau SYN dans un paquet consomme un numéro de séquence
  - drapeau ACK est vrai uniquement si le TPDU contient un numéro d'acquit valide
  - horloge TCP incrémentée toutes les 4 µsec et à chaque établissement de connexion

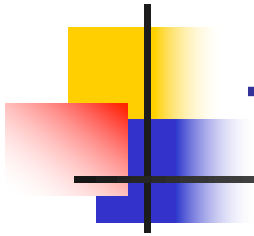




TCP : Gestion des connexions

Ouverture de connexion

- L'extrémité demandant l'ouverture de la connexion, est le *client*. Il émet un segment SYN (où le bit SYN est fixé à 1) spécifiant le numéro de port du *serveur* avec lequel il veut se connecter.
- Il expédie également un numéro de séquence initial *N* (ISN : *Initial Sequence Number*)
  - ISN est généralement dérivé de l'horloge de l'hôte
  - exemple : un tic par 4 microsecondes
- Cette phase est appelée ouverture active et «consomme» un numéro de séquence.



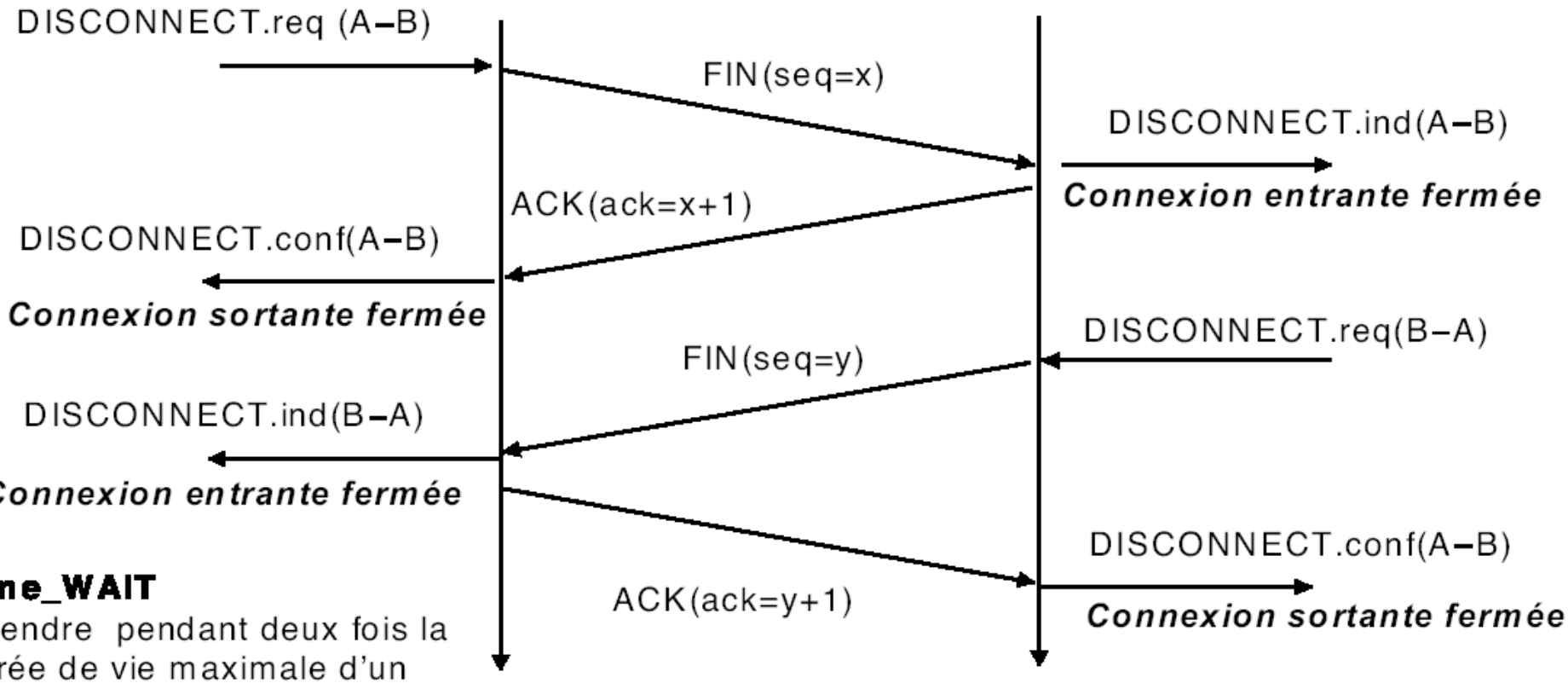
TCP : Gestion des connexions

Ouverture de connexion

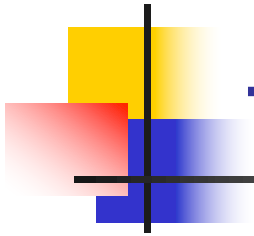
- Le serveur répond en envoyant un segment dont les bits ACK et SYN sont fixés à 1. Ainsi, dans un même segment il acquitte le premier segment reçu avec une valeur de ACK=*X*+1 et il indique un numéro de séquence initial.
- Cette phase est appelée ouverture passive.
- Le client TCP doit évidemment acquitter ce deuxième segment en renvoyant un segment avec ACK= *Y*+1

TCP : Gestion des connexions

Fermeture de connexion



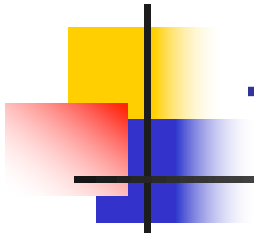
Time_WAIT
Attendre pendant deux fois la durée de vie maximale d'un paquet dans le réseau au cas où un TPDU FIN serait à nouveau reçu



TCP : Gestion des connexions

Fermeture de connexion

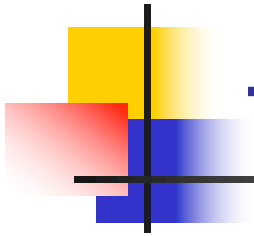
- Terminaison normale :
  - fin demandée par l'une des extrémités (flag FIN = 1)
  - acquittement et attente de la réciproque
- Terminaison brutale :
  - envoi de la primitive ABORT (flag RST = 1)
  - toutes les transmission ou réceptions sont interrompues et les tampons sont vidés



TCP : Gestion des connexions

Fermeture de connexion

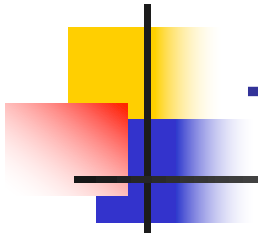
- Terminaison normale :
  - La terminaison d'une connexion peut être demandée par n'importe quelle extrémité et se compose de deux *demi-fermetures* puisque des flots de données peuvent s'écouler simultanément dans les deux sens.



TCP : Gestion des connexions

Fermeture de connexion

- L'extrémité qui demande la fermeture (le client dans émet un segment où le bit FIN est fixé à 1 et où le numéro de séquence vaut *X*.
- Le récepteur du segment l'acquitte en retournant un ACK=*X*+1 et informe l'application de la demi-fermeture de la connexion.



TCP : Gestion des connexions

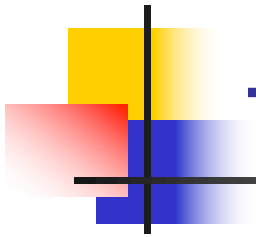
Fermeture de connexion

- À partir de là, les données ne peuvent plus transiter que dans un sens (de l'extrémité ayant accepté la fermeture vers l'extrémité l'ayant demandée), et dans l'autre seuls des accusés de réception sont transmis.
- Quand l'autre extrémité veut fermer sa demi-connexion, elle agit de même que précédemment ce qui entraîne la terminaison complète de la connexion.

Fonctionnalités de TCP

TCP : Type de Transmission

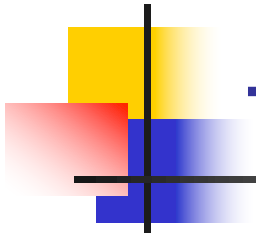
- **Bidirectionnel**
  - *full duplex :* composée de deux flots de données indépendants et de sens contraire.
  - Il est cependant possible d'inclure dans l'en-tête de segments TCP d'une communication de A vers B des informations relatives à la communication de B vers A. Cette technique de superposition (piggybacking) permet de réduire le trafic sur le réseau
- **Point-à-point**
  - envoi de messages entre les deux hôtes d'extrémité
  - Pas de diffusion



TCP : transfert de données

TCP : Type de Transmission

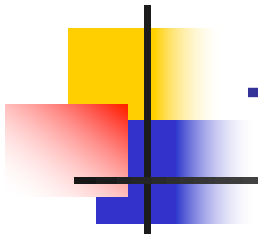
- **Transport d'un flot d'octet**
  - *byte stream oriented*
  - Tout au long de la connexion, TCP échange un *flux d'octets* sans qu'il soit possible de séparer par une marque quelconque certaines données.
  - Le contenu des octets n'est pas du tout interprété par TCP, c'est donc aux applications d' extrémité de savoir gérer la structure du flot de données.



TCP : transfert de données

TCP : Type de Transmission

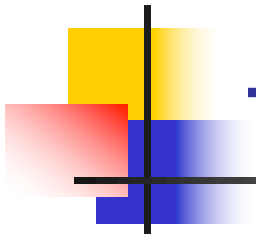
- protocole orienté **octets**
  - ne préserve pas les frontières de message
- pas de contrôle sur les émissions :
  - données envoyées immédiatement ou bufférisées
- récupération de données **urgentes**
  - exemple : pour le traitement d'un CTRL-C



Types de transfert TCP

TCP : Type de Transmission

- Le ***transfert de données*** de TCP est de deux types :
  - le transfert **interactif** dans lequel chaque segment transporte très peu d'octets, voire un seul,
  - et le transfert **en masse** où chaque segment transporte un maximum d'octets.



Types de transfert TCP

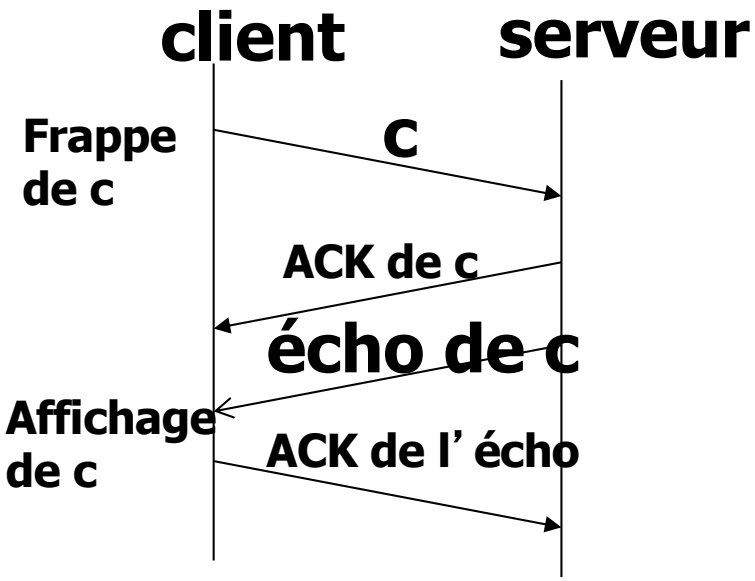
TCP : Type de Transmission

- Un exemple de transfert interactif est celui généré par la commande **rlogin** lancée depuis un client vers un serveur.
  - Dans ce cas de figure, tous les caractères tapés par l'utilisateur sur le client sont envoyés vers le serveur en utilisant un caractère par segment, et ils sont ensuite renvoyés en sens inverse par le serveur pour un écho sur l'écran du client.
  - Tous les segments échangés dans ce cas là ont leur bit PSH fixé à 1. Or, tout segment doit être acquitté dans un sens comme dans l'autre; ce qui devrait amener au diagramme d'échanges illustré dans le a) de la figure

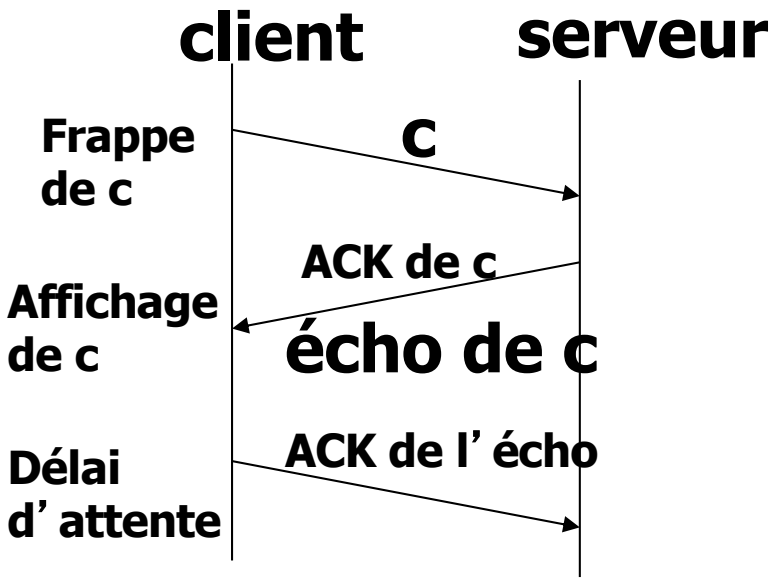
Types de transfert TCP

TCP : Type de Transmission

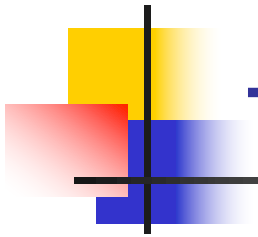
- exemple de transfert interactif (commande rlogin)



(a)



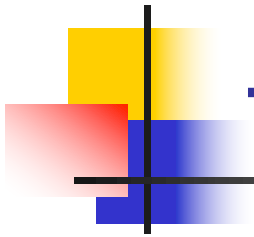
(b)



TCP : Acquittement retardé

TCP : Type de Transmission

- En fait, TCP gère ce type d'échange avec la procédure **d'acquittement retardé** qui consiste à envoyer l'acquittement en l'incluant dans un segment qui transporte également des données (b)
- Pour cela l'acquittement est généralement retardé de 200ms dans le but d'attendre d'éventuelles données à transmettre.
- Cependant, dans ce contexte la frappe de la commande date sur le client provoquerait quand même l'échange de 15 datagrammes IP de 41 octets, pour transporter à chaque fois seulement 1 octet utile.



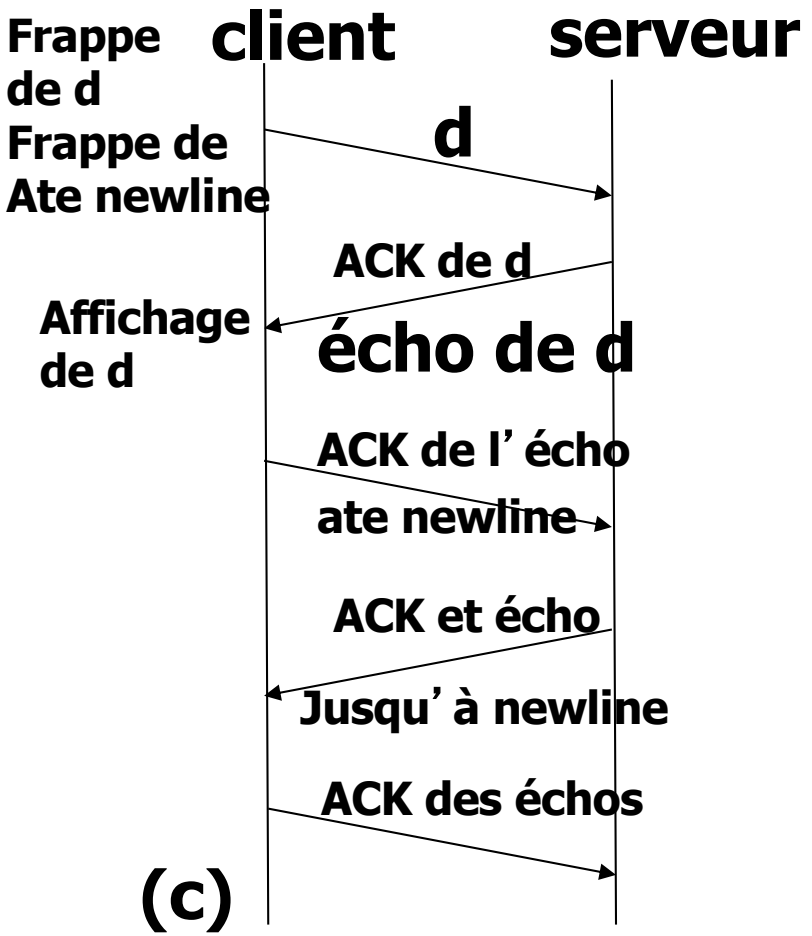
TCP : Acquittement retardé

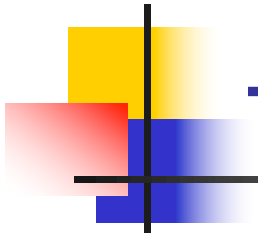
TCP : Type de Transmission

- Ce type de situation est peu gênant sur un LAN mais devient très vite préjudiciable au bon fonctionnement d'un WAN.
- Une solution à ce problème est **l'algorithme de Nagle** (RFC 896) qui spécifie qu'une connexion TCP ne peut avoir seulement un petit segment non encore acquitté.
  - Ainsi, si un réseau a un fort débit et n'est pas du tout chargé la procédure sera celle du b) de la figure
  - Par contre, dès que le temps de transfert est non négligeable, les acquittements vont arriver plus lentement que la frappe des caractères par l'utilisateur du poste client.

TCP : Acquittement retardé

- Dans ce cas, la couche TCP du client va accumuler de petits volumes de données (quelques caractères) et les envoyer dans le même segment TCP diminuant ainsi considérablement le nombre d'échanges comme illustré dans le c) de la figure

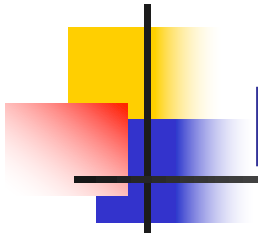




TCP : Acquittement retardé

TCP : Type de Transmission

- Dans les cas où de petits messages doivent être remis sans délai (mouvement de souris dans le cas d'un serveur X) l'algorithme de Nagle doit être invalidé pour donner une impression de temps réel.



Fonctionnalités de TCP

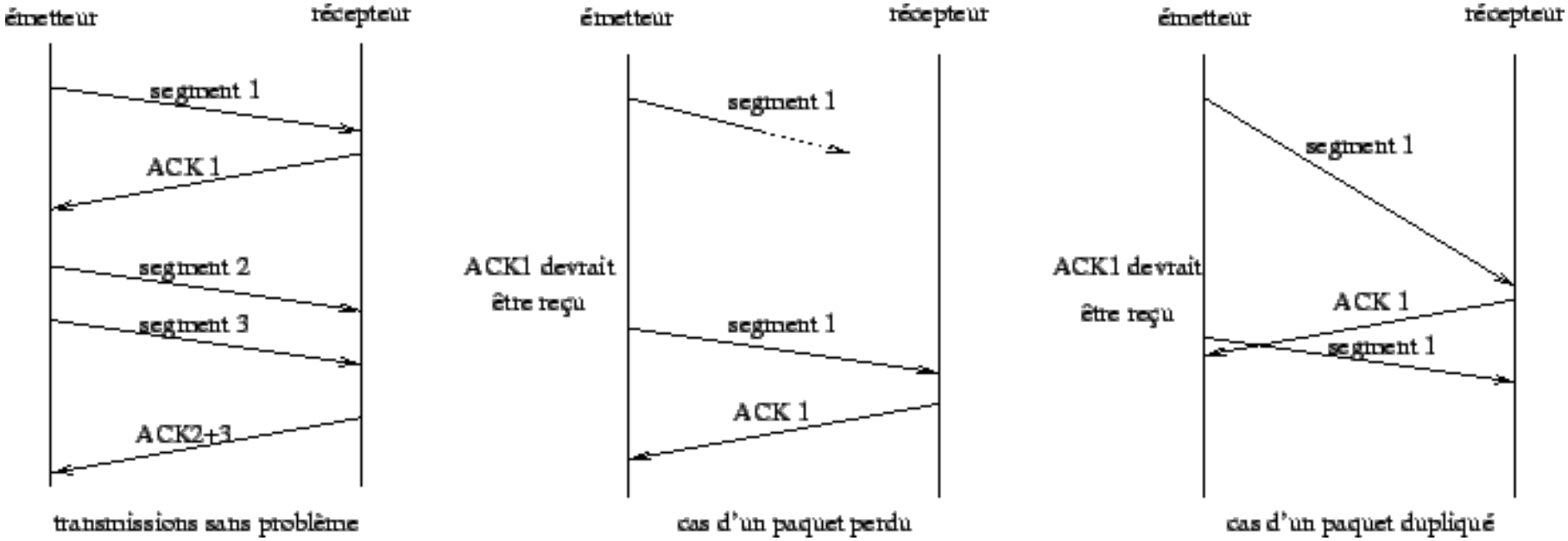
TCP : Fiabilisation et Robustesse

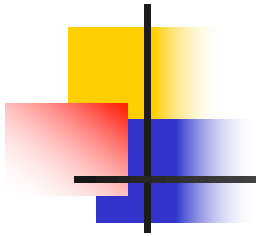
- **Contrôle d'erreur de transmission**
  - détection d'erreurs sur les données reçues
- **Ordonnancement**
  - détection des paquets dé-séquencés
  - détection des duplications
- **Fiabilisation de la transmission**
  - acquittements positifs et cumulatifs
  - retransmissions sur temporisation
  - ordonnancement

UNIVERSITÉ SORBONNE PARIS NORD

Fonctionnalités de TCP

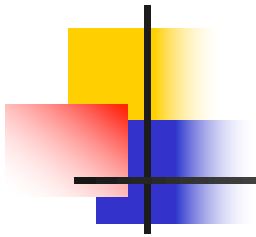
- La ***fiabilité*** fournie par TCP consiste à remettre des datagrammes, sans perte, ni duplication, alors même qu'il utilise IP qui lui est un protocole de remise non fiable.
- Ceci est réalisé à l'aide de la technique générale de l'accusé de réception (*ACK*)





Fonctionnalités de TCP

- Chaque segment est émis avec un numéro qui va servir au récepteur pour envoyer un accusé de réception.
  - Ainsi l'émetteur sait si l'information qu'il voulait transmettre est bien parvenue à destination.
- De plus, à chaque envoi de segment, l'émetteur arme une temporisation qui lui sert de délai d'attente de l'accusé de réception correspondant à ce segment.
  - Lorsque la temporisation expire sans qu'il n'ait reçu de ACK, l'émetteur considère que le segment s'est perdu et il le réexpédie.

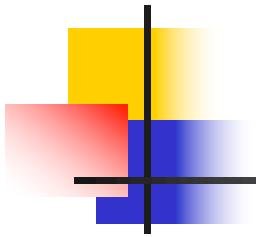


Fonctionnalités de TCP

TCP : Fiabilisation et Robustesse

- Mais il se peut que la temporisation expire alors que le segment a été transmis sans problème, par exemple suite à un engorgement de réseau ou à une perte de l'accusé de réception correspondant.

- Dans ce cas, l'émetteur réémet un segment alors que c'est inutile. Mais le récepteur garde trace des numéros de segments reçus, donc il est apte à faire la distinction et peut éliminer les doublons.



Fonctionnalités de TCP

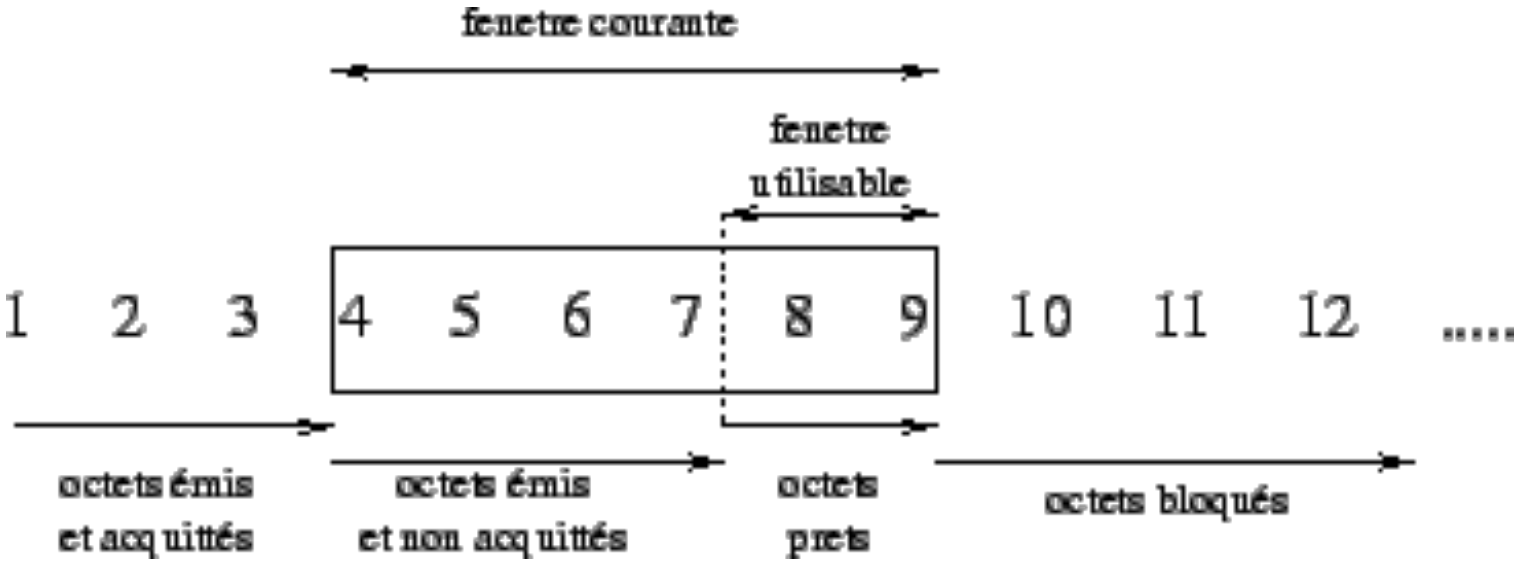
TCP : Adaptation et Dynamisme

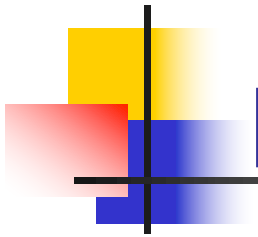
- **Contrôle de flux :**
  - dans le cas d'un transfert de données en masse, TCP utilise la technique de la *fenêtre glissante* (Fenêtre coulissante) pour contrôler le flux des échanges
    - ***sliding window***
    - Ceci est primordial quand un micro ordinateur communique avec un gros ordinateur, sinon le tampon d'entrée du micro sera très vite saturé.
      - Ceci consiste en un contrôle de flux de *bout en bout*. Mais il s'agit aussi de réguler le trafic en fonction de la charge des routeurs et du débit des réseaux traversés.

Fonctionnalités de TCP

TCP : la fenêtre glissante

- l'ensemble d'un flux de données unidirectionnel d'une machine A vers une machine B est constitué d'une séquence d'octets tous numérotés individuellement
- La fenêtre glissante va consister à fixer quels sont les octets appartenant à ce flux que A peut émettre



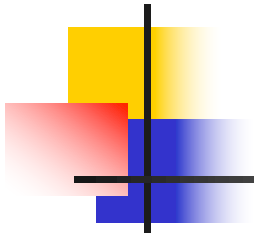


Fonctionnalités de TCP

TCP : la fenêtre glissante

- Dans cet exemple la fenêtre couvre les octets de 4 à 9, car la taille de la fenêtre courante est 6 et que tous les octets jusqu'au troisième inclus ont été émis et acquittés

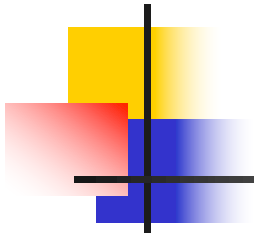
- A tout instant TCP calcule sa fenêtre utilisable qui est constituée des octets présents dans la fenêtre et non encore envoyés. Ces octets sont généralement immédiatement transmis.



Fonctionnalités de TCP

TCP : la fenêtre glissante

- Pour le flot de A vers B, la taille de la fenêtre est contrôlée par B qui envoie dans chacun de ses accusés de réception la taille de la fenêtre qu'il désire voir utiliser.
  - Si la demande exprime une augmentation, A déplace le bord droit de sa fenêtre courante et émet immédiatement les octets qui viennent d'y entrer.
  - Si la demande exprime une diminution, il est déconseillé de déplacer réellement le bord droit de la fenêtre vers la gauche. Ce rétrécissement est opéré lors des glissements de la fenêtre vers la droite avec l'arrivée des accusés de réception.



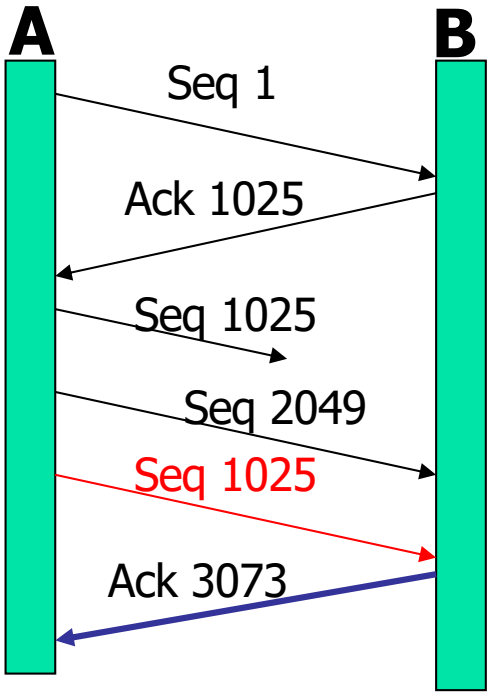
Fonctionnalités de TCP

- **Acquittement cumulatif**
  - TCP n'acquitte pas un à un chaque segment qu'il reçoit,
  - mais l'ensemble du flot de données jusqu'à l'octet K-1 en envoyant un acquittement de valeur K

Fonctionnalités de TCP

acquittement cumulatif

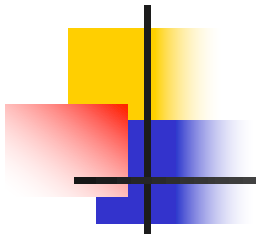
- Par exemple, dans une transmission de 3 segments de A vers B, si les octets de 1 à 1024 sont reçus correctement, alors B envoie un ACK avec la valeur 1025.



A vers B

3 segments :

- 1 - 1024
- 1025 - 2048
- 2049 - 3072

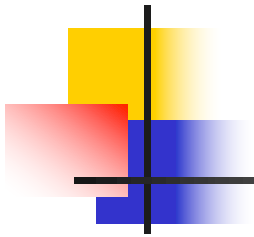


Fonctionnalités de TCP

acquittement cumulatif

- Puis, si le segment suivant contenant les octets de 1025 à 2048 se perd et que B reçoit d'abord correctement le segment des octets de 2049 à 3072, B n'enverra pas d'accusé de réception positif pour ce troisième segment.
- Ce n'est que lorsque B recevra le deuxième segment, qu'il pourra envoyer un ACK avec la valeur 3073, que A interprétera comme l'acquittement des deux derniers segments qu'il a envoyés.

- On appelle cela un acquittement cumulatif.



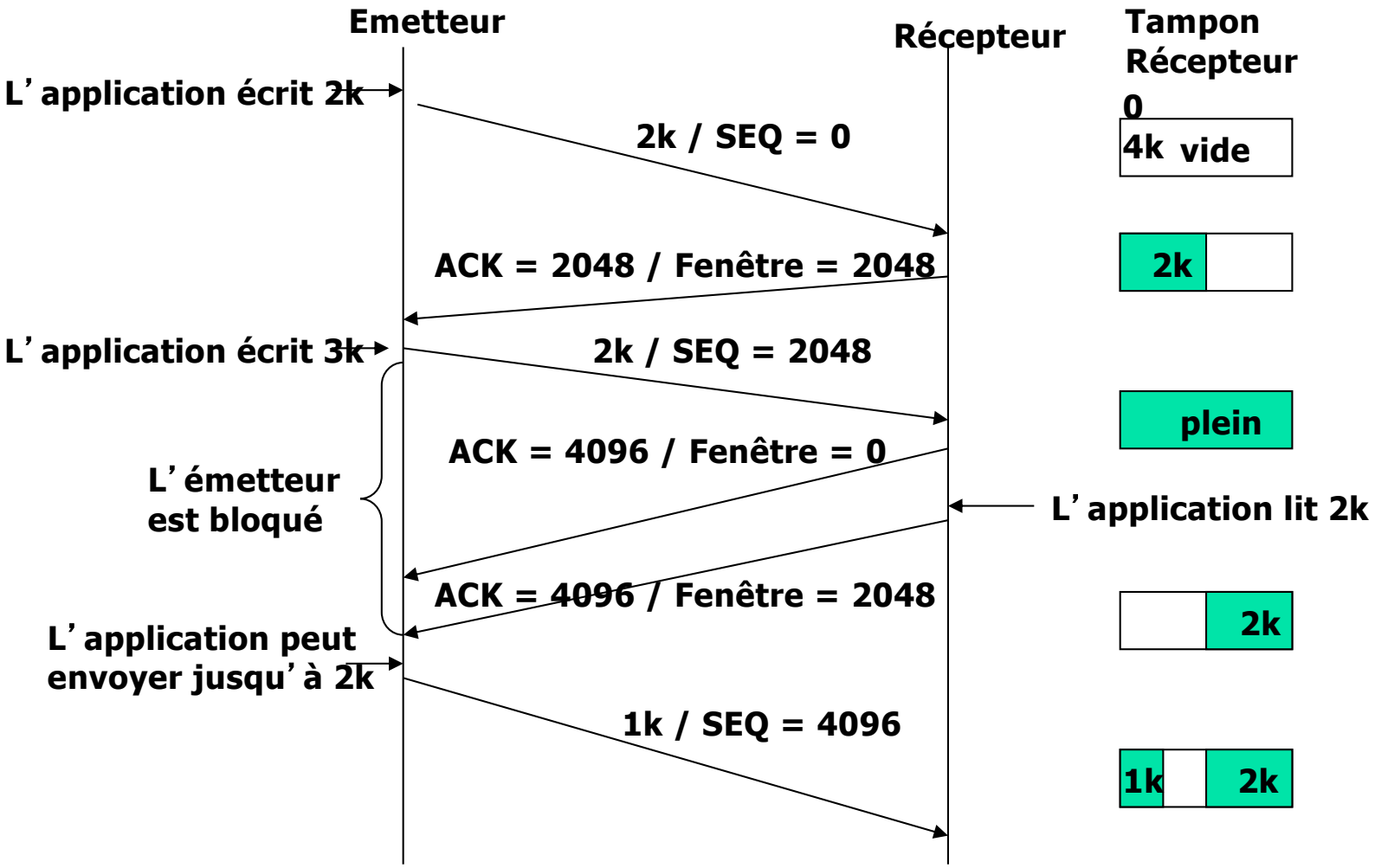
Fonctionnalités de TCP

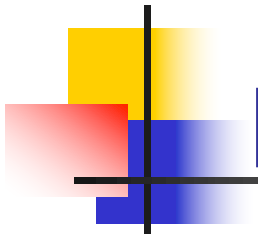
Politique de transmission TCP

- **Gestion des fenêtres**
  - Découplage des acquittement et de l'envoi de données
  - Emission retardée pour remplir les buffers (mais PUSH possible pour l'interactivité)
  - Acquittements retardé (ex: 500 ms) pour attendre un segment de retour (sinon création d'un segment vide de 41 octets spécialement)

UNIVERSITÉ SORBONNE PARIS NORD

Fonctionnalités de TCP





Fonctionnalités de TCP

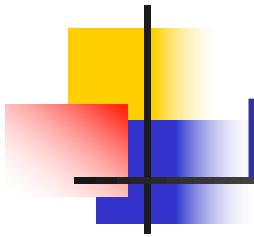
Temporisation TCP

- TCP utilise de nombreux temporisateurs :
  - Temporisateur de persistance
    - pour débloquer la situation après une fenêtre d ‘émission réduite à zéro et une ouverture perdue
  - Temporisateur d'inactivité
    - indique de vérifier la présence de l'autre extrémité
  - Temporisateur de déconnexion
    - deux fois la durée de vie des paquets

Fonctionnalités de TCP

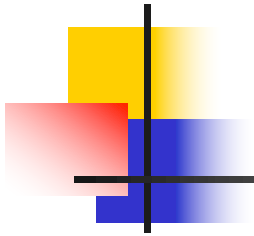
Temporisation TCP

- Temporisateur de retransmission :
  - Démarré à chaque envoi d'un paquet en attendant son acquittement
  - calcul du RTT :
  - RTT = a*RTT + (1-a)M avec ex: a=0.2
  - retransmission au bout de b*RTT avec ex: b=2
- Solution choisie par IP
  - mesurer le rtt moyen (rtt_moy)
  - estimer la variance du rtt (rtt_var)
  - Temporisateur = rtt_moy + 4 rtt_var



Fonctionnalités de TCP

- **urgence**
  - envoi d'un paquet immédiat sans contrôle de flux
  - certaines applications demandent que les données soient émises immédiatement, même si le tampon n'est pas plein
    - Pour cela, elles utilisent le principe du ***push*** pour forcer le transfert. Les données sont alors émises avec un bit marquant cela pour que la couche TCP réceptrice du segment remette immédiatement les données à l'application concernée.
- **Contrôle de congestion**
  - deux algorithmes :
    - *slow start*
    - c*ongestion avoidance*



Fonctionnalités de TCP

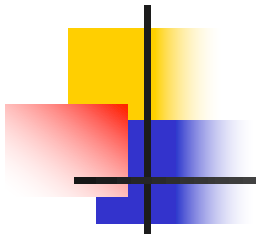
Multiplexage et Ports

- Pour permettre à plusieurs tâches d'une même machine de communiquer simultanément via TCP, le protocole définit un ensemble d'adresses et de ports pour la machine.

- Pour avoir un service TCP il faut créer deux points de connexion : **sockets**
  - Accès à TCP à travers des points d'accès (appelés ***sockets*** sous Unix BSD)

Fonctionnalités de TCP

- *socket* est associée à une adresse IP et un **port**
  - port = TCP TSAP = numéro sur 16 bits
- connexion TCP liée à 2 *sockets*
  - *(Socket1-source, sockets-destination) =*
  - *(Ip-Source+Port-Source, Ip-Destination+Port-Destination)*
- une *socket* être lié à plusieurs connexions



Fonctionnalités de TCP

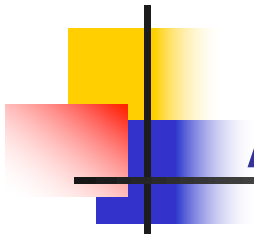
Multiplexage et Ports

- L'affectation des ports aux processus est établie par chaque ordinateur. Cependant, il semble judicieux de réserver certains numéros de ports pour des services caractérisés et souvent utilisés. Ces services standard pourront alors être atteints via ces ports réservés

- Ports : valeur locale
  - 0 à 255 : *well kown ports*
  - 0 à 1023 : ports système
  - 1024 à 65535 : port utilisateurs

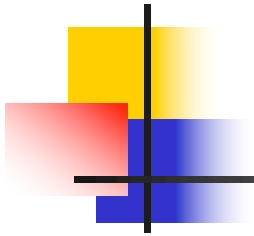
Quelques *well-know TCP ports* et les services associés

numéro	nom	description
7	ECHO	Echo
11	USERS	Active Users
13	DAYTIME	Daytime
20	FTP-DATA	File Transfert Data
21	FTP-CTRL	File Transfert Control
22	SSH	Secure Shell
23	TELNET	Telnet
25	SMTP	Simple Mail
53	DOMAIN	Domain Name Server
67	BOOTPS	Bootstrap Server
68	BOOTPC	Bootstrap Client
69	TFTP	Trivial FTP
79	FINGER	Finger
102	ISO-TSAP	ISO Transport



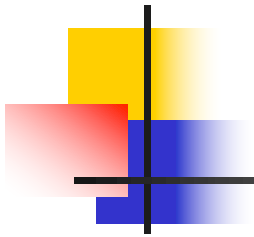
Applications TCP

- Les applications suivantes reposent typiquement sur TCP
  - Web
  - connexion à distance (TELNET, RLOGIN, SSH)
  - transfert de fichiers (FTP, rcp, scp, sftp)
  - protocole de routage externe (BGP)
  - messageries instantanées (IRC, ICQ, AIM ...)



2. Introduction à UDP

- Le protocole UDP (rfc 768) utilise IP pour acheminer, d'un ordinateur à un autre, en mode non fiable des datagrammes qui lui sont transmis par une application
- UDP n'utilise pas d'accusé de réception et ne peut donc pas garantir que les données ont bien été reçues
- Il ne réordonne pas les messages si ceux-ci n'arrivent pas dans l'ordre dans lequel ils ont été émis et il n'assure pas non plus de contrôle de flux.



Introduction à UDP

- Il se peut donc que le récepteur ne soit pas apte à faire face au flux de datagrammes qui lui arrivent. C'est donc à l'application qui utilise UDP de gérer les problèmes de perte de messages, duplications, retards, déséquencement, ...
- Cependant, UDP fournit un service supplémentaire par rapport à IP, il permet de distinguer plusieurs applications destinatrices sur la même machine par l'intermédiaire des *ports*.
  - Un port est une destination abstraite sur une machine identifié par un numéro qui sert d'interface à l'application pour recevoir et émettre des données.

Introduction à UDP

- Un datagramme UDP est contenu dans un datagramme IP
- Chaque datagramme émis par UDP est encapsulé dans un datagramme IP en y fixant à 17 la valeur du protocole



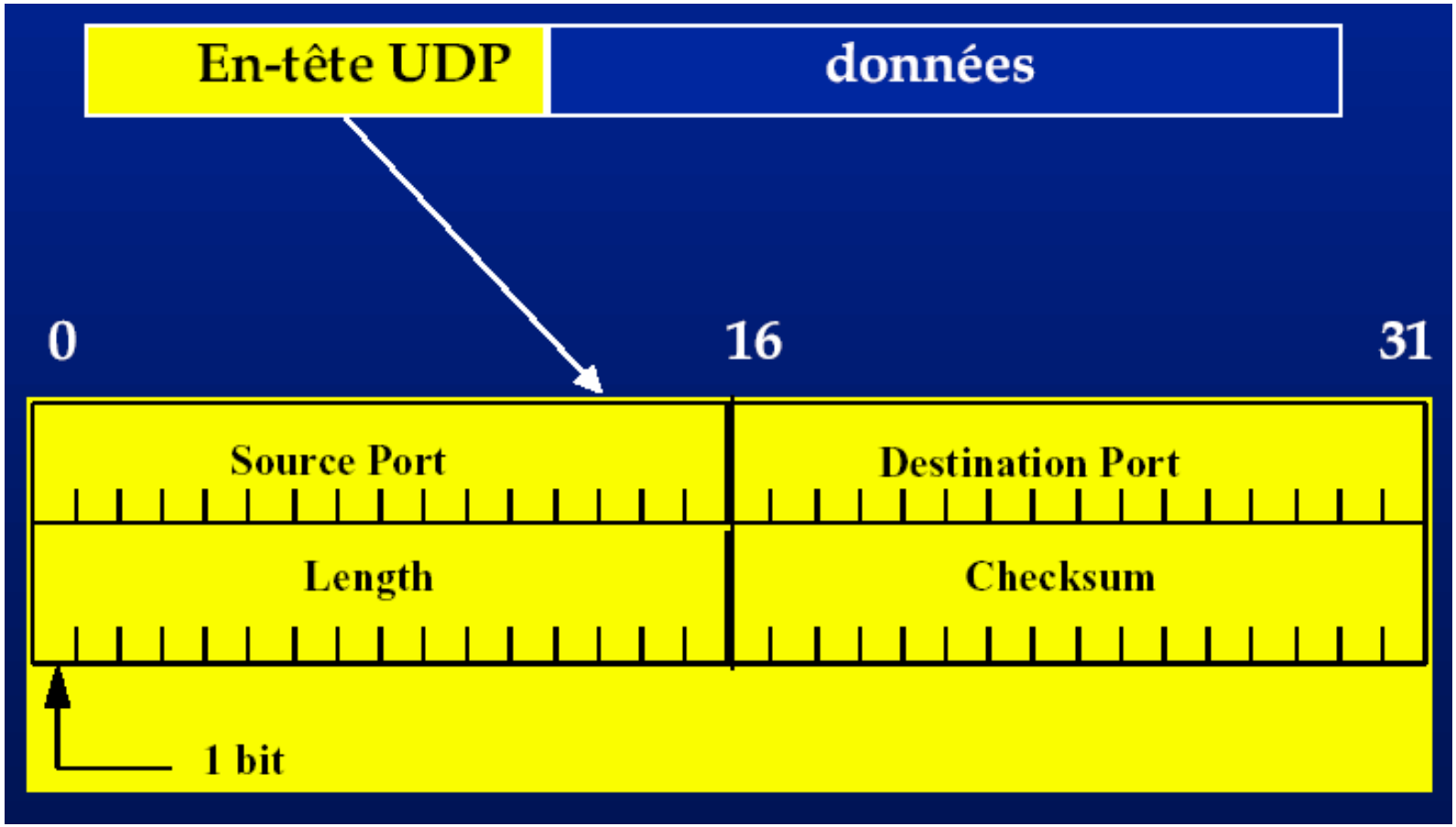
Datagramme UDP

- Le format détaillé d'un datagramme UDP est donné dans la figure

Port UDP source	Port UDP destination
longueur	checksum
données	

Structure d'un Datagramme UDP

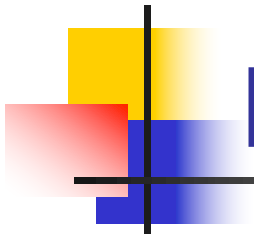
En-tête UDP


En-tête UDP
données
0
16
31
Source Port
Destination Port
Length
Checksum
1 bit

En-tête UDP

- Les *numéros de port* (chacun sur 16 bits) identifient les processus émetteur et récepteur.
- Le champ *longueur* contient sur 2 octets la taille de l'en-tête et des données transmises.
  - Puisqu'un datagramme UDP peut ne transmettre aucune donnée la valeur minimale de la longueur est 8.

Port UDP source	Port UDP destination
longueur	checksum



En-tête UDP

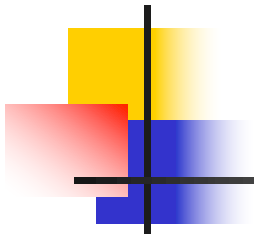
- Le *checksum* est un total de contrôle qui est optionnel car il n'est pas indispensable lorsque UDP est utilisé sur un réseau très fiable. S'il est fixé à 0 c'est qu'en fait il n'a pas été calculé.

- De manière précise, UDP utilise l'en-tête et les données mais également une *pseudo-entête* pour aboutir à l'ensemble décrit

En-tête UDP

- Champs utilisés pour le calcul du checksum UDP

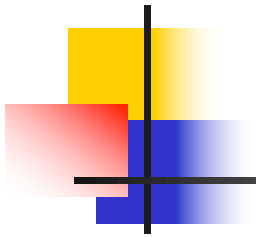
Adresse IP souce	
Adresse IP destination	
Port UDP source	Port UDP destination
longueur	checksum
données	
	Bit de bourrage



En-tête UDP

- Cette pseudo en-tête comprend les adresses IP source et destination du datagramme ainsi qu'un éventuel octet de bourrage pour aboutir à un nombre d'octets total pair.

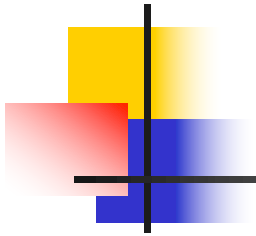
- À partir de cet ensemble, le total de contrôle est calculé de la même manière que dans le cas du datagramme IP
  - Si le résultat donne un checksum nul, son complément à 1, c'est-à-dire 65535 (16 bits à 1), est en fait placé dans la zone de contrôle



En-tête UDP

- Ce détail permet d'éviter la confusion avec le checksum nul qui indique qu'il n'a pas été calculé. Précisons enfin que la pseudo en-tête et l'octet de bourrage ne sont pas transmis et qu'ils n'interviennent pas dans le calcul du champ longueur.

- À la réception UDP utilise l'adresse IP de destination et l'adresse IP émettrice inscrite dans l'en-tête du datagramme IP pour calculer, de la même manière qu'à l'émission, une somme de contrôle qui permettra d'assurer que le datagramme est délivré sans erreur et à la bonne machine.

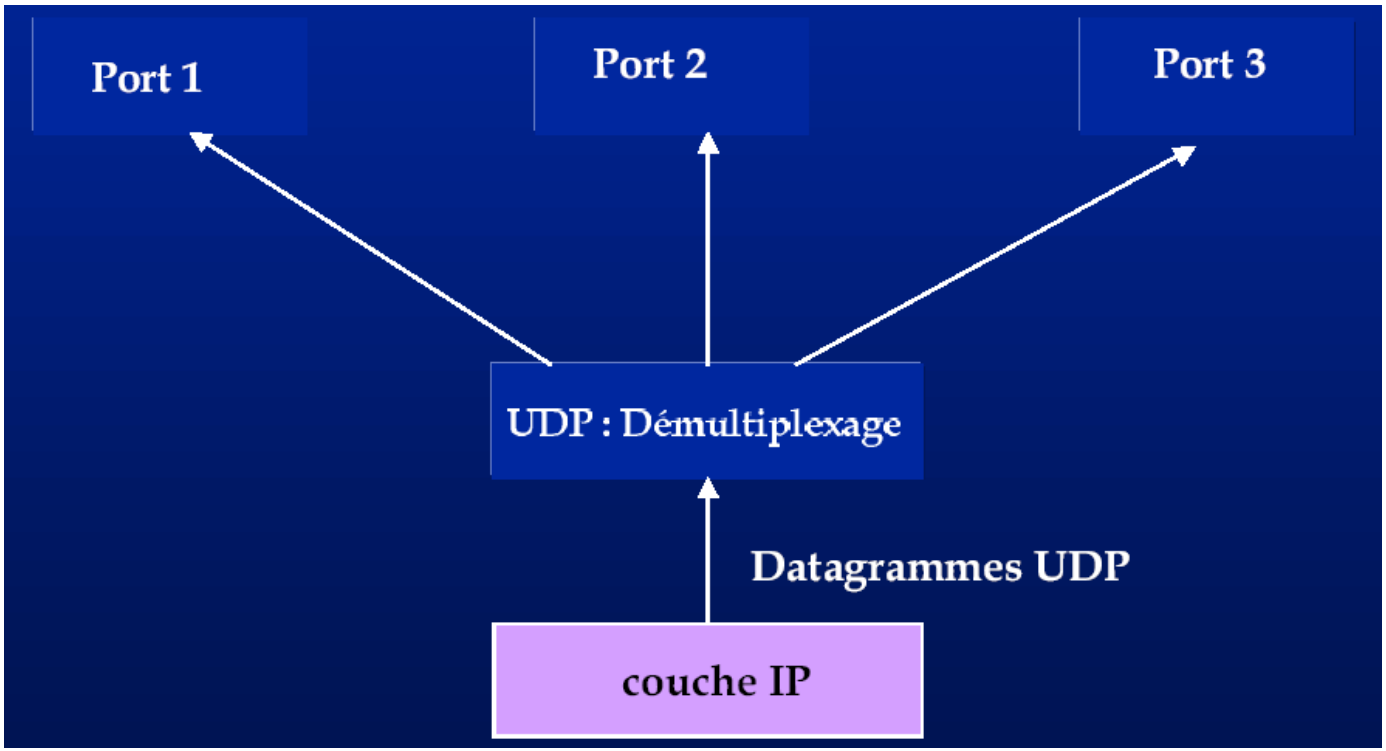


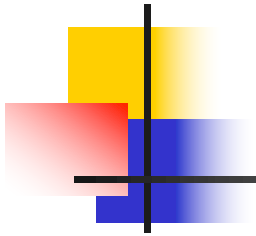
En-tête UDP

- Si une erreur de transmission est détectée, le datagramme UDP est détruit «en silence»
- Sinon, UDP oriente les données du datagramme vers la file d'attente associée au numéro de port destination pour que l'application associée à celui-ci puisse les y lire.

Protocole UDP

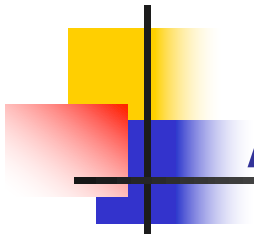
- Par rapport à IP, UDP rajoute l'information indiquant le service





Protocole UDP

- Ne fait pas
  - Mode connecté
  - Retransmission si erreur ou perte
  - Séquencement
  - Contrôle de flux
- C’ est un protocole de transport non fiable
- Utilisé pour la diffusion
  - Dès que le multicast est nécessaire



Applications UDP

- Les applications suivantes reposent typiquement sur UDP :
  - résolution de noms (DNS)
  - serveur de fichiers distants (NFS)
  - protocole de routage (RIP)
  - administration du réseau (SNMP)
  - diffusion multimédia, streaming (propriétaire)
  - téléphonie sur IP
  - etc.

et toutes les solution multicast